

Table of Contents

[PatternLanguageDefTopic](#)

[platpg.5.01](#)

[platpg.5.02](#)

[platpg.5.03](#)

[platpg.5.04](#)

[platpg.5.05](#)

[platpg.5.06](#)

[platpg.5.07](#)

[platpg.5.08](#)

[platpg.5.09](#)

[platpg.5.10](#)

[platpg.5.11](#)

[platpg.5.12](#)

[platpg.5.13](#)

[plheaderstatements.2.01](#)

[plheaderstatements.2.02](#)

[plheaderstatements.2.03](#)

[plheaderstatements.2.04](#)

[plheaderstatements.2.05](#)

[plheaderstatements.2.06](#)

[plheaderstatements.2.07](#)

[plheaderstatements.2.08](#)

[plheaderstatements.2.09](#)

[plheaderstatements.2.10](#)

[plheaderstatements.2.11](#)

[plheaderstatements.2.12](#)

[plheaderstatements.2.13](#)

[plheaderstatements.2.14](#)

[plheaderstatements.2.15](#)

[plheaderstatements.2.16](#)

[plheaderstatements.2.17](#)

[plheaderstatements.2.18](#)

[plheaderstatements.2.19](#)

[plheaderstatements.2.20](#)

[plheaderstatements.2.21](#)

[plheaderstatements.2.22](#)

[plheaderstatements.2.23](#)

[plmicrocode.4.01](#)

[plmicrocode.4.02](#)

[plmicrocode.4.03](#)

[plmicrocode.4.04](#)

[plmicrocode.4.05](#)

[plmicrocode.4.06](#)

[plmicrocode.4.07](#)

[plmicrocode.4.08](#)

[plmicrocode.4.09](#)

[plmicrocode.4.10](#)

[plmicrocode.4.11](#)

[plmicrocode.4.12](#)

[plmicrocode.4.13](#)

[plmicrocode.4.14](#)

[plmicrocode.4.15](#)

[plnotes](#)

[ploverview.1.2](#)

[ploverview.1.3](#)

[ploverview.1.4](#)

[ploverview.1.5](#)

[ploverview.1.6](#)

[ploverview.1.7](#)

[ploverview.1.8](#)

[plvectors.3.01](#)

- [plvectors.3.02](#)
- [plvectors.3.03](#)
- [plvectors.3.04](#)
- [plvectors.3.05](#)
- [plvectors.3.06](#)
- [plvectors.3.07](#)
- [plvectors.3.08](#)
- [plvectors.3.09](#)
- [plvectors.3.10](#)
- [plvectors.3.11](#)
- [plvectors.3.12](#)
- [plvectors.3.13](#)
- [plvectors.3.14](#)
- [plvectors.3.15](#)

PatternLanguageDefTopic

	
---	---

About Pattern Language

About Pattern Language

This section includes an overview of the pattern language and covers other information relevant to the entire pattern file.

The following topics are available:

- | |
|--|
| <ul style="list-style-type: none">Summary of Pattern Language Statements |
|--|

- [Pattern File Format Requirements](#)

- [Pattern File Filename and Pathname](#)

- [Syntax Notation](#)

- [Comments in Pattern Files](#)

- [Preprocessor Statements](#)

- [Reserved Words in Pattern Files](#)

In addition to this overview section, the following sections are also available:

- [Pattern Header and Setup Statements](#)

- [Modules, Vectors, and Vector Data](#)

- [Microcode \(Opcodes and Control Bits\)](#)

- [Automatic Test Pattern Generation \(ATPG\) Syntax](#)

- [Sample Pattern](#)



platpg.5.01

<	>
-----------------	-----------------

Automatic Test Pattern Generation (ATPG) Syntax

Automatic Test Pattern Generation (ATPG) Syntax

Starting in IG-XL V7.40.00, ASCII test pattern language (.atp) files includes syntax to encapsulate automatic test pattern generation (ATPG) information in a compiled IG-XL binary pattern (.pat) files. One use of this ATPG information is to populate STDF V4-2007.1 Pattern Sequence Records (PSR). You can process this language using the IG-XL pattern compiler. You can also use the command line interface utility patinfo to view the ATPG information in a binary pattern file. These ATPG statements also offer a way to track the versions of the pattern source file, translation tools, and other parts of pattern creation and compilation.

This section describes the ATPG header and setup statements and how to view this ATPG information from a binary pattern. It contains these topics:

- [ATPG Header Statements](#)
- [ATPG Setup Statements](#)
- [ATPG Syntax in a ATP Pattern File Example](#)
- [Viewing ATPG Information in a Binary Pattern File](#)

<	>
-----------------	-----------------

--	--	--

platpg.5.02

<	>
-----------------	-----------------

Automatic Test Pattern Generation (ATPG) Syntax : ATPG Header Statements

ATPG Header Statements

ATPG header statements uniquely identify the source ATPG file and provide information about the translation software used to convert that source ATPG data into an IG-XL source ASCII file. ATPG header statements include:

- | | |
|---|----------------------------|
| • | atpg_source_file_name |
| • | atpg_source_file_timestamp |
| • | atpg_source_file_id |
| • | atpg_tool_id |
| • | atpg_translator_id |
| • | atpg_translation_timestamp |

☒ **Note:**

ATPG source can be in many different formats, but a commonly used format is Standard Test Interface Language (STIL), which is also known as IEEE 1450. Many examples in this section use references to STIL, but this functionality is not limited to the STIL format.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

platpg.5.03

<	>
---	---

Automatic Test Pattern Generation (ATPG) Syntax : ATPG Header Statements : atpg_source_file_name

atpg_source_file_name
This statement defines the originating ATPG source file name (for example, STIL).
Format
atpg_source_file_name = "FileName"
Parameters

FileName	String. The ATPG source file name.
----------	------------------------------------

Usage
The FileName string is encapsulated in the binary pattern file exactly as specified. Define this source file name based on the specific ATPG generation and diagnostic flow and tools.
You can include this statement only once in the .atp file header.
Example
atpg_source_file_name = "Scan_TP1.stil"

<	>
---	---

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

platpg.5.04

<	>
---	---

Automatic Test Pattern Generation (ATPG) Syntax : ATPG Header Statements :

atpg_source_file_timestamp

atpg_source_file_timestamp

This statement defines the time stamp of the originating source ATPG file.

Format

atpg_source_file_timestamp = "TimeStamp"

Parameters

TimeStamp	String. The ATPG source file time stamp.
-----------	--

Usage

The TimeStamp string is encapsulated in the binary pattern file as specified. Define this string's format and content based on the specific ATPG generation and diagnostic flow and tools.

You can include this statement only once in the .atp file header.

For STIL files, this statement typically corresponds to the STIL header block's "Date" field.

Example

atpg_source_file_timestamp = "Mon Apr 16 19:34:41 2010"

<	>
---	---

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

platpg.5.05

<	>
---	---

Automatic Test Pattern Generation (ATPG) Syntax : ATPG Header Statements : atpg_source_file_id

atpg_source_file_id

This statement provides an additional field for uniquely identifying the ATPG source file.

Format

atpg_source_file_id = "FileID"

Parameters

FileID	String. The ATPG source file unique identifier or description.
--------	--

Usage

Use this statement to provide additional file identification details (for example, descriptions, revision, checksum, file size, and so on). The FileID string is encapsulated in the binary pattern file as specified. Define this string's format and content based on the specific ATPG generation and diagnostic flow and tools.

You can include this statement only once in the .atp file header.

For STIL files, this statement typically corresponds to the STIL header block's "Title" field.

Example

atpg_source_file_id = "V3.0: Scan pattern #1"

<	>
---	---

platpg.5.06

<	>
---	---

Automatic Test Pattern Generation (ATPG) Syntax : ATPG Header Statements : atpg_tool_id

atpg_tool_id

This statement defines the specifics of the ATPG tools used to create the source ATPG (STIL) file.

Format

atpg_tool_id = "ToolID"

Parameters

ToolID	String. The ATPG tool identifier.
--------	-----------------------------------

Usage

Use this parameter to include a tool name and a uniquely identifying revision or time stamp. This ToolID string is encapsulated in the binary pattern file as specified. Define this string's format and content based on the specific ATPG generation and diagnostic flow and tools.

You can include this statement only once in the .atp file header.

For STIL files, this statement typically corresponds to the STIL header block's "Source" field.

Example

atpg_tool_id = "FastScan v8.2008_4.10-snapshot_2008.10.21_05.00"

<	>
---	---

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

platpg.5.07

<	>
---	---

Automatic Test Pattern Generation (ATPG) Syntax : ATPG Header Statements : atpg_translator_id

atpg_translator_id

This statement defines the specifics of the ATPG-to-ATE translator tool used to create the IG-XL pattern file.

Format

atpg_translator_id = "TranslatorID"

Parameters

TranslatorID	String. The ATPG-to-ATE translator tool identifier.
--------------	---

Usage

Use this statement to include a translator tool name and a uniquely identifying revision or time stamp. This TranslatorID string is encapsulated in the binary pattern file as specified. Define this string's format and content based on the specific ATPG generation and diagnostic flow and tools. You can include this statement only once in the .atp file header.

Example

atpg_translator_id = "ATEGen Version 1.0.21 [win x86_32] 27-Apr-2009"

<	>
---	---

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

platpg.5.08

<	>
---	---

Automatic Test Pattern Generation (ATPG) Syntax : ATPG Header Statements :

atpg_translation_timestamp

atpg_translation_timestamp

This statement defines the time stamp of when the ATPG-to-ATE translation occurred.

Format

atpg_translator_id = "TransTimeStamp"

Parameters

TransTimeStamp	String. The ATPG-to-ATE translation time stamp.
----------------	---

Usage

This TransTimeStamp string is encapsulated in the binary pattern file as specified. Define this string's format and content based on the specific ATPG generation and diagnostic flow and tools. Use the same time stamp format for both the translation process and the ATPG source file.

You can include this statement only once in the .atp file header.

Example

atpg_translation_timestamp = "Tue Apr 17 11:22:37 2010"

<	>
---	---

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

platpg.5.09

<	>
-----------------	-----------------

Automatic Test Pattern Generation (ATPG) Syntax : ATPG Setup Statements

ATPG Setup Statements

ATPG setup statements map ATPG file contents to IG-XL pattern file contents and capture any modifications that occur during the translation process. The setup section uses two basic high level constructs:

- `atpg_pinmap`
- `atpg_info`

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

platpg.5.10

<

>

Automatic Test Pattern Generation (ATPG) Syntax : ATPG Setup Statements : atpg_pinmap

atpg_pinmap

This statement supports optionally mapping individual ATPG pin names (not group names) to different pin names in the IG-XL environment.

Format

```
atpg_pinmap =  
{  
    IGXLPinName : ATPGPinName,  
    IGXLPinName : ATPGPinName,  
    ...  
    IGXLPinName : ATPGPinName  
}
```

Parameters

IGXLPinName	String. The pin names in the IG-XL environment. This parameter does not support pin group names.
ATPGPinName	String. The ATPG pin names. This parameter does not support pin group names.

Usage

All IG-XL tools (for example, DataTool sheets, PatternTool, and debug displays) use IG-XL pin names. When you use this statement, however, the ATPG pin name can be used either directly (for example, proprietary ASCII scan fail logs) or indirectly (for example, using the STDF V4-2007 NMR record) when reporting scan fail data.

The IG-XL pin naming rules and restrictions do not apply to the ATPG pin name.

Each IG-XL pattern source file can have only one atpg_pinmap statement, and the mapping it defines is assumed to apply to all pattern modules contained in that file. Note that UltraFLEX supports multiple modules per file.

You can remap a subset of pins. Like the scan_pins syntax, the last entry in the list of remapped pins does not include a comma.

For STIL files, individual ATPG pin names map to pin names specified in the files "Signals" block.

Example

```
atpg_pinmap =  
{  
SO0 : SO[0],  
SO1 : SO[1],  
SO2 : SO[2],  
SO3 : SO[3]  
}
```

<	>	
	2015 Teradyne, Inc. All rights reserved.	

platpg.5.11

<

>

Automatic Test Pattern Generation (ATPG) Syntax : ATPG Setup Statements : atpg_info

atpg_info

This statement specifies how ATPG blocks map to IG-XL pattern data and modules and how cycle counts are affected when modules are merged into a single pattern burst on the test system.

Format

```
atpg_info [ModuleName] =  
{  
  [atpg_source_id = "IDName"];  
  [log_first_cycle] = TesterCycleNum;  
  [log_last_cycle] = TesterCycleNum;  
}
```

Parameters

ModuleName	Optional String. The module name. This parameter allows you to specify data on a per-module basis. This parameter is unneeded if the IG-XL pattern file has only one module or set of pattern data. If you do not specify a module name and the IG-XL pattern file contains multiple pattern modules, then all parameter values set in this statement are applied to all modules within that pattern file.
atpg_source_id = IDName	Optional String. This statement specifies the ATPG block identifier (IDName) in the source data. For STIL files, this block identifier is the PatternExec name, if specified. Note that STIL only requires a name when multiple PatternExec blocks exist in the file.

<code>log_first_cycle = TesterCycleNum</code>	Optional Long. This statement describes how the first cycle of the referenced module maps to the tester PatGen cycle.
<code>log_last_cycle = TesterCycleNum</code>	Optional Long. This statement describes how the last cycle of the referenced module maps to the tester PatGen cycle.

Usage

An .atpg file can contain up to one of these constructs for each module in the file. All constructs are contained in the file's setup section.

All parameters are optional and end with a semicolon, so that you can specify any combination of them within the atpg_info construct.

This statement allows you to specify how ATPG blocks map to PatGen cycle counts depending on whether pattern bursts are separated or merged on the test system.

Many ATPG tools can create multiple blocks of scan patterns in a single ATPG source file (for example, multiple PatternExec blocks in STIL format). In the pattern translation process, those blocks can become separate IG-XL pattern files or separate modules within a single IG-XL pattern file. Note that UltraFLEX supports multiple modules per file.

You can selectively translate a subset of the available blocks in an ATPG source file using this statement. Additionally, within the test program, you can define the program's execution setup so that the output patterns are run as individual patterns during a burst or merged into a single pattern burst. Note that a pattern burst can include a single pattern module or several pattern modules that are burst together by using a pattern set with the burst mode enabled (Pattern Sets sheet's Burst column is set to yes).

Many ATPG generators support logging various pattern index and tester cycle counts and include them in the ATPG source file. For example, Mentor FastScan allows you to output the information for the specific scan burst index, ATPG vector, and resulting tester cycle to STIL as an annotation as shown below.

```
Pattern Scan_Test_1 {
...
//Pattern:13 Vector:62 TesterCycle:404
Ann {* Pattern:13 Vector:62 TesterCycle:404 *}
"pattern 13":
V {
  _pi_=001101101100110110111010011100100111011111001001101001010011111111111110
      100001100001011100010100011011110;
  _po_=XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
      XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
      XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX; }
Macro "scan_grp1" {
```

```
"_chain1_scan_in1_" = 0001110001101111000;  
"_chain1_scan_out1_" = LLLHHHLLLHHLHHHHLXX;  
...  
}
```

Additionally, many translators support calculating the number of resulting tester cycles required for the translated output pattern files.

By default, you typically program the log_first_cycle to zero and the log_last_cycle to the number of tester cycles required to execute the pattern, where that number can come from the source file or be calculated by the translator. You may need to adjust this cycle count mapping, however, when you are executing multiple pattern modules in a single burst.

For example, if you have three separate patterns with 1000 cycles each in a pattern set with the burst mode enabled so that they run as a single pattern burst, then IG-XL has cycle counts 0 to 2999. The diagnostic tool, however, expects fails for each pattern to be in the range of 0-999. In this case, you can use the log_first_cycle and log_last_cycle statements to indicate the offset for each pattern module as follows:

Example PatGen Cycle Count Mapping for ATPG Blocks

Pattern Module	Cycle Start (log_first_cycle)	Cycle End (log_last_cycle)
1	0	999
2	1000	1999
3	2000	2999

Example

```
atpg_info vm_module_1 =  
{  
  atpg_source_id = "PatExec_1";  
  log_first_cycle = 0;  
  log_last_cycle = 121000;  
}  
  
atpg_info vm_module_2 =  
{  
  atpg_source_id = "PatExec_2";  
  log_first_cycle = 121001;  
  log_last_cycle = 234567;  
}
```

<		>	
	2015 Teradyne, Inc. All rights reserved.		

platpg.5.12

<	>
---	---

Automatic Test Pattern Generation (ATPG) Syntax : ATPG Syntax in a ATP Pattern File Example

ATPG Syntax in a ATP Pattern File Example

The example below shows ATPG syntax used in an ATP pattern file.

```
digital_inst = hsdm;
```

```
opcode_mode = single;
```

```
cdl_cache = yes;
```

```
compressed = no;
```

```
import tset GrossFunc_ts, ScanShift_ts;
```

```
scan_type = X3;
```

```
atpg_translator_id = "ATEGen Version 1.1.6 [win x86_32] 06-Jun-2010";
```

```
atpg_tool_id = "FastScan v8.2008_4.10-snapshot_2008.10.21_05.00";
```

```
atpg_translation_timestamp = "Sun Oct 24 17:25:01 2010";
```

```
atpg_source_file_name = "Example_ATPG_Source.stil";
```

```
atpg_source_file_id = "This is a dummy STIL file";
```

```
atpg_source_file_timestamp = "Fri Jun 25 09:30:00 2008";
```

```
atpg_info ScanTP1a_vm = {
```

```
atpg_source_id = "PatExec_2";
```

```
log_first_cycle = 0;
```

```
log_last_cycle = 11144;
```

```
}
```

```
atpg_pinmap = {
```

```
SO0 : SO[0],
```

```
SO1 : SO[1],
```

```
SO2 : SO[2],
SO3 : SO[3],
PO0 : Block(0).PO{0},
PO1 : Block(0).PO{1},
PO2 : Block(0).PO{2},
PO3 : Block(0).PO{3}
}

scan_pins = {
SI0,
SO0,
SI1,
SO1,
SI2,
SO2,
SI3,
SO3
}
//
vm_vector ScanTP1a_ym ( $tset, CHAIN3:S, CHAIN2:S, CHAIN1:S, CHAIN0:S, PI_ALL:S, PO_ALL:S,
CLK_ALL:S)
...
```

platpg.5.13

<

>

Automatic Test Pattern Generation (ATPG) Syntax : Viewing ATPG Information in a Binary Pattern File

Viewing ATPG Information in a Binary Pattern File

Once an IG-XL pattern is compiled, you can view the included ATPG information using the `patinfo` command-line utility. Run this utility from the command prompt and provide arguments to indicate what information it should output. Arguments related to viewing the ATPG information are as follows:

-switches

Outputs all switch information, including the ATPG related parameter information contained in the pattern header section. For example:
COMPILER SWITCH TABLE

...
atpg_translator_id statement STRING ATEGen Version 1.1.6 [win x86_32] 06-Jun-2010
atpg_translation_timestamp statement STRING Sun Oct 24 17:25:01 2010
atpg_tool_id statement STRING FastScan v8.2008_4.10-snapshot_2008.10.21_05.00
atpg_source_file_name statement STRING Example_ATPG_Source.stil
atpg_source_file_id statement STRING This is a dummy STIL file
atpg_source_file_timestamp statement STRING Fri Jun 25 09:30:00 2008
...

-atpgpinmap

Outputs the information contained in the `atpg_pinmap` construct. For example:
ATE ATPG Pin Map:

ATE PinName ATPG PinName

PO0 Block(0).PO{0}

PO1 Block(0).PO{1}
PO2 Block(0).PO{2}
PO3 Block(0).PO{3}
SO0 SO[0]
SO1 SO[1]
SO2 SO[2]
SO3 SO[3]

-atpginfo Outputs the information contained in the atpg_info constructs. For example:
ATPG Parameters:

atpg_translator_id: ATEGen Version 1.1.6 [win x86_32] 06-Jun-2010
atpg_translation_timestamp: Sun Oct 24 17:25:01 2010
atpg_tool_id: FastScan v8.2008_4.10-snapshot_2008.10.21_05.00
atpg_source_file_name: ScanTP1.stil
atpg_source_file_id: TP1 with no fails inserted
atpg_source_file_timestamp: Fri Jun 25 09:30:00 2008

ATPG Information:

Name: ScanTP1a_vm
Log first cycle: 0
Log last cycle: 11144
Source ID:

For more information on this utility, see [Pattern Information Utility](#).

plheaderstatements.2.01

<

>

Pattern Header and Setup Statements

Pattern Header and Setup Statements

Pattern header statements contain information required to compile a pattern, such as the digital instrument for which the pattern is being compiled and the pin map where pins are defined.

Header statements include:

- Digital Instrument Statement
- Pin Map Specification
- Compiler Control Statements
- Import Tset or Label Statement

Setup statements define special digital pin and scan pin setups, plus any analog instruments that are used.

Setup statements include:

- Pin_setup Statement
- Pin Setups and Valid Character Sets
- Instruments Statement
- Scan_pins Statement

UltraFLEX pattern files also support header and setup statements used to encapsulate automatic test pattern generation (ATPG) information in binary pattern files. For more information, see [Automatic Test Pattern Generation \(ATPG\) Syntax](#).

< >	
	2015 Teradyne, Inc. All rights reserved.

plheaderstatements.2.02



Pattern Header and Setup Statements : Digital Instrument Statement

Digital Instrument Statement

Digital Instrument

This statement specifies the digital instrument for which a pattern file is to be compiled.

You can specify the digital instrument either by putting a `digital_inst` statement into the pattern file, as described here, or by using the compile-time option of the Pattern Compiler.

For more information, see [Pattern Compiler Options](#) in the Pattern Compiler section.

Syntax

For UltraFLEX, the digital instruments currently supported are:

- [HSDM](#) (in pattern files, [HSD1000](#) and [UltraPin800](#) instrument channels are represented as `hsdm`, not `hsd_m`, `hsdi`, or `hsd800`):

```
digital_inst = hsdm;
```

- [HSDMQ](#) (in pattern files, [UltraPin1600](#) instrument channels are represented as `hsdmq`, not `hsdu` or `hsd1600`)

```
digital_inst = hsdmq;
```

When compiling a pattern that contains [MTO](#), the `digital_inst` statement must use `hsdm`, even if you are using the [UltraPin1600](#) instrument. The pattern compiler returns an error if you use `hsdmq`.

☑ **Note:**

You can run [HSDM](#) patterns using a mixture of [HSD1000](#), [UltraPin800](#), and [UltraPin1600](#) digital channels, but you can only run [HSDMQ](#) patterns when assigning pins to [UltraPin1600](#) channels.

- [HSD4G](#) (in pattern files, both [UltraPin4000](#) and [HPM](#) instrument channels are represented as `hsd4g`)

digital_inst = hsd4g;

- Serial6G:

digital_inst = Serial6G;

- UltraSerial10G:

digital_inst = Serial10G;

For more information, see [HSDM](#), [HSDMQ](#), and [HSD4G Controls](#), [Serial6GControls](#), and [Serial10G Controls](#).

<

>

2015 Teradyne, Inc. All rights reserved.

plheaderstatements.2.03



Pattern Header and Setup Statements : Pin Map Specification

Pin Map Specification

The pattern compiler must have access to pin map information, to determine whether a name is for a pin or a pin group of pins, or how many pins are in a group. Pin map information is contained on a Pin Map sheet of an IG-XL workbook.

You can specify the workbook and a Pin Map sheet name either by putting a pinmap_workbook and a pinmap_sheet statement into the pattern file, as described here, or by using the compile-time options of the Pattern Compiler.

For more information, see [Pattern Compiler File Specifications](#) in the Pattern Compiler section.

Syntax

pinmap_workbook = "filename";

pinmap_sheet = "sheetname";

where:

filename	The name of an IG-XL workbook (extension .xls).
sheetname	The name of a Pin Map sheet (the name on the tab) in the workbook.

When Required

- If a workbook contains only one Pin Map sheet, the pinmap_sheet specification is not required.
- If a workbook contains more than one Pin Map sheet, the pinmap_sheet specification is required. If there is no pinmap_sheet specification, an error is generated.

Notes

- If both the pinmap_workbook statement and the compile-time option are used, the compile-time option overrides the statement without generating a warning message. The same is true of the statement and compile-time option for pinmap_sheet.
- The pattern compiler gives an error if a pin or group name found in the vector statement pinlist cannot be found in the Pin Map sheet.
- The pattern compiler does not check that pins declared as input in the pin map use only drive data, or that pins declared as output in the pin map use only receive data.

plheaderstatements.2.04

<	>
---	---

Pattern Header and Setup Statements : Compiler Control Statements

Compiler Control Statements

The pattern language supports compiler control statements for several options. You can also specify these options at compile time using the pattern compiler. Not all compile-time selections have corresponding pattern file control statements. If a compile-time selection conflicts with a control statement, the compile-time selection takes precedence, and the control statement is ignored.

The pattern language includes two kinds of compiler control statements. Some apply to all digital instruments. Others apply only to specific digital instruments. The statements included here are:

- General Compiler Control Statements
- HSDM, HSDMQ, and HSD4G Controls
- Serial6GControls
- Serial10G Controls

For more information, see [Pattern Compiler](#) in the Pattern Compiler section.

<	>
---	---

plheaderstatements.2.05

<	>
-----------------	-----------------

Pattern Header and Setup Statements : Compiler Control Statements : General Compiler Control Statements

General Compiler Control Statements

The following compiler control statements apply to all digital instruments:

- | | |
|---|-----------------------|
| • | compressed |
| • | import_all_undefineds |
| • | lock |
| • | opcode_mode |
| • | output_filename |
| • | post_processor |
| • | save_comments |
| • | scan_type |
| • | version |

compressed
compressed = no | yes;

Whether the pattern compiler should compress the compiled output file. The default is yes. If compression is enabled, the pattern compiler compresses vector data internally into discrete blocks.

`import_all_undefineds`

`import_all_undefineds = no | yes;`

How the pattern compiler should handle references to undefined tset names or to labels that are still unresolved at the end of compilation: import them without giving a warning or error (yes) or give errors (no). The default is no. See [Import Tset or Label Statement](#).

This control statement is for use with any preprocessor that generates the names on the fly. Using this control statement is not recommended since errors in names are not caught until pattern load time.

`lock`

`lock = no | yes;`

Whether to lock the compiled pattern file. The default is no. A locked pattern file cannot be opened by pattern tools or reverse-compiled. See [Pattern Locking](#) in the pattern compiler section.

`opcode_mode`

In hsdm patterns, which use the HSD1000 or UltraPin800 instruments, the valid opcode modes are:

`opcode_mode = single | dual;`

where single means there can be an opcode on every vector and dual means there can be an opcode on every other vector. The default is dual.

In hsdmq patterns, which use the UltraPin1600 instrument, the valid opcode modes are:

`opcode_mode = single | dual | quad;`

where single means there can be an opcode on every vector, dual means there can be an opcode on every other vector, and quad means there can be an opcode on every fourth vector. The default is dual.

In hsd4g patterns, which use the UltraPin4000 instrument, the valid opcode modes are:

`opcode_mode = by1 | by16 | by16iio;`

where by1 means there can be an opcode on every vector and by16 or by16iio means there can be an opcode on every 16th vector. The default is by16.

In hsd4g patterns, which use the HPM instrument, the valid opcode modes are:

`opcode_mode = by1 | by1_tof | by16;`

where by1 or by1_tof means there can be an opcode on every vector and by16 means there can be an opcode on every 16th vector. The default is by16.

`output_filename`

`output_filename = "filename";`

The output filename. The default is the input filename, with the extension replaced by .pat.

See [Pattern File Filename and Pathname](#) for restrictions on the pattern file name.

`post_processor`

```
post_processor = "filename";
```

```
post_processor_args = "args";
```

What post-processor to run on the input .atp file.

These statements let you run a customized post-processor on the input pattern file (.atp) at the end of the compilation of the pattern. The `post_processor` statement specifies the pathname of the post-processor executable. Its value must be an enquoted string. The `post_processor_args` statement specifies the string of additional arguments, apart from the .atp filename (which is automatically passed to the post-processor), that should be passed to the post-processor executable. Its value must also be an enquoted string.

When you specify the post-processor, it is run on the .atp file by the pattern compiler under following rules:

- The post-processor takes the input .atp file as its input. If you run a C/C++ preprocessor (specified using the Pattern Compiler interface), the post-processor takes the output of the C/C++ pre-processor as its input.
- The pattern compiler runs the post-processor at the end of the compilation of the pattern file.
- The pattern compiler passes the pathname of the input pattern file (or the output file from the C/C++ preprocessor) to the post-processor as its first argument. Then the additional arguments as programmed in the `post_processor_args` statement.
- If the post-processor fails, the pattern compiler removes the output .pat file. A success compilation therefore requires a successful post-processing.
- If the post-processor is an executable with a .bat or .cmd extension, the pattern compiler automatically invokes the MS-DOS command line interpreter to execute it.
- If the post-processor is an executable with .pl extension, the pattern compiler automatically invokes the Perl interpreter to execute it.

The pattern compiler expects the following from the user-provided post-processor:

- The post-processor should not modify the contents of its input file.

- When errors occur during post-processing, the post-processor should exit with nonzero exit code.

save_comments

save_comments = no | yes;

Whether to save comments in the file. See [Comments in Pattern Files](#).

If you omit this statement, the default for the .atp file is not to save comments. The Pattern Compiler interfaces can override the setting for the .atp file. See [Saving Comments in a Compiled File](#).

scan_type

scan_type = type;

The method for packing scan data into memory. For HSD1000 and UltraPin800 (hsdm) and UltraPin1600 (hsdmq) digital instruments, the options for type are:

x3, x6, x12, x24	These scan types specify scan packing methods that allow 2-bit scan chains. The number after x specifies packing efficiency: data of 3/6/12/24 scan cycles is packed into one parallel vector.
x2, x4, x8	These scan types specify scan packing methods allow 3-bit scan chains. The number after x specifies packing efficiency: data of 2/4/8 scan cycles is packed into one parallel vector.
extended	This scan type allows 3-bit scan chains. Data of 12 scan cycles is packed into one parallel vector.
parallel	This scan type allows expanding scan vectors into parallel vectors. Data of one scan cycle is packed into one parallel vector. Scan data is converted to parallel data. This type allows either 2-bit or 3-bit scan chains. No compression is achieved with this scan type. This consumes vector memory at the same rate as any other functional test vector. Note that if you are doing scan testing, but are not using the SCAN hardware packing in your application, use the scan pattern syntax and the parallel compiler option, rather than using a pattern translation process, to create parallel vectors. This approach allows you to use the scan hardware packing in the future if your scan depth requirements change. Moreover, it retains scan chain information in the pattern file, which can be used by the pattern debug tool and other scan diagnostic tools.

For the HSDM digital instruments (HSD1000 and UltraPin800), the default is x3.

For the HSDMQ digital instrument (UltraPin1600), the default is x3. Note that SCAN is only supported in HSDMQ patterns when using single or dual opcode modes. SCAN is not supported in

quad mode.

 Note:

This compiler control statement is not used on the HSD4G digital instrument (UltraPin4000 and HPM).

For more information, see `scan_type` in the SCAN section.

version

`version = "text";`

A user-defined string to be placed in the pattern binary, intended for customer version-tracking.

<	>	
	2015 Teradyne, Inc. All rights reserved.	

plheaderstatements.2.06

	
---	---

Pattern Header and Setup Statements : Compiler Control Statements : HSDM, HSDMQ, and HSD4G Controls

HSDM, HSDMQ, and HSD4G Controls

The following controls can be used only when `digital_inst` specifies `hsdm`, `hsdmq`, and `hsd4g`.

`allow_mto_dash`

`allow_mto_dash = no | yes;`

Whether the default pin setup supports the use of a dash (-) to indicate runtime repeat in an MTO pattern.

In MTO patterns (that is, patterns with MTO programming in an instruments statement), this option controls whether you can select the default pin setup that supports the pin state "-" for runtime repeat. In non-MTO patterns, the option has no effect.

Setting the option to `yes` lets you use the dash. The default is `no`.

This option is not used on the UltraPin4000 and HPM instruments.

`cdl_cache`

`cdl_cache = no | yes;`

Whether to use channel data latch (CDL) caching. CDL caching is hardware technique for compressing vector data. It uses CDL structure and a small hardware cache of vector data.

Turning on CDL caching (`yes`) results in smaller binary pattern file, while turning it off (`no`) results in shorter compilation time. The default is `yes`.

`check_set_msb`

`check_set_msb = no | yes;`

Whether to perform a check to see if the pattern meets the `set_msb` requirement. The `opcode` `set` or `set_infinite` must be preceded by at least one `set_msb` or `set_msb_infinite` opcode in a pattern module. Setting this option to `no` means the compiler does not check this requirement. The default is `yes`.

`init_pattern`

`init_pattern = "pathname\patternfile.pat[:patternmodule]" | "patternmodule" | "patternset";`

A pattern module, pattern file, or pattern set that is to be executed before executing the current pattern file.

You can have initialization patterns that are required to be executed before you can run the patterns that actually perform a test. This feature lets you create pattern sets that contain only the patterns performing the test without having to include the initialization vectors.

You must include the path for the initialization pattern file or module. If the initialization pattern is a module that resides in the current pattern file, you need not include the name of the pattern file. A pattern set can also be used as an initialization pattern.

You can specify an initialization pattern on the pattern compiler command line. This lets you use a single initialization pattern for many pattern files, without having to hard code the pattern name in the file.

For more information, see [Initialization Pattern](#).

time_domain

time_domain = "time-domain-name" | tlSystem | tlBestFit;

The name of the time domain to be used for the pins in the pattern.

The specified time domain must contain all pins in the pattern. Pattern compiler verifies that all pins in the pattern file belong to the specified time domain.

If the time domain is not specified, the pattern compiler searches the pin map for the smallest time domain, that is, the one with the smallest number of pins that still contains all the pins in the pattern, and it uses that time domain for the pattern. If it cannot find any time domain that contains all pins in the pattern, it uses the System time domain for the pattern. The System time domain is a single time domain that is comprised of all of the pins defined in the test program.

You can also use one of these keywords, instead of the time domain name:

- **tlSystem**: Assign a pattern to the default System time domain in a multiple time domain pin map.
- **tlBestFit**: Assign a pattern to the best fit time domain in a multiple time domain pin map. This option allows you to override a non-default time domain name in the ATP file.

plheaderstatements.2.07

<	>
-----------------	-----------------

Pattern Header and Setup Statements : [Compiler Control Statements](#) : [Serial6GControls](#)

Serial6GControls

The following controls can be used only when `digital_inst` specifies Serial6G.

`check_vm_min_size`

`check_vm_min_size = no | yes;`

Whether to check the minimum number of vectors within each module. If this option is enabled (yes), the vector size must be ≥ 1024 ; otherwise, an error message is returned.

The default is yes.

`check_vm_mod_size`

`check_vm_min_size = no | yes;`

Whether to check that the total number of vectors within each module is multiple of 1024. If this option is enabled (yes), the number of vectors should be multiple of 1024; otherwise, an error message is returned.

The default is yes.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

plheaderstatements.2.08

<	>
---	---

Pattern Header and Setup Statements : Compiler Control Statements : Serial10G Controls

Serial10G Controls

The following controls can be used only when `digital_inst` specifies Serial10G.

`check_oob_size`

`check_oob_size = no | yes;`

Whether to check that total number of out of band (OOB) regions within each module is a multiple of 10. If this option is enabled (yes), each OOB region must start and end on a 10-bit symbol boundary; otherwise, an error message is returned.

The default is no.

`check_vm_mod_size`

`check_vm_min_size = no | yes;`

Whether to check that the total number of vectors within each module is either a multiple of 2560, or in the case of a pattern set with only one pattern or the last module in a pattern set, a multiple of 40. If this option is enabled (yes), the number of vectors should be a multiple of 2560 or 40; otherwise, an error message is returned.

The default is no.

<	>
---	---

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

plheaderstatements.2.09

<	>
---	---

Pattern Header and Setup Statements : Import Tset or Label Statement

Import Tset or Label Statement

Tset Names

All tset (time set) names used in a pattern file must first be imported using an import tset statement. This allows the pattern compiler to detect misspellings.

The syntax for importing tset names is:

```
import tset tsetname, tsetname, ... ;
```

For more information, see [Tset \(Time Set\) Data](#).

Labels

Any external labels (labels that are referenced but not defined in a pattern file) must first be imported. Use a separate import statement for each type of label. You can define multiple labels of a type in a single import statement, but each type of label requires a separate import statement.

The following import label statements are supported:

```
import label labelname, labelname, ... ;
```

Import any labels exception for subroutine labels.

```
import subr labelname, labelname, ... ;
```

Import subroutine labels. See [Subroutines](#).

```
import stop subr labelname, labelname, ...;
```

Import labels indicating stop subroutines used by pattern modules in the pattern file.

For more information, see [Vector Labels](#).

Case Sensitivity

Labels and tset names are not case-sensitive.

<	>
---	---

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

plheaderstatements.2.10

<

>

Pattern Header and Setup Statements : Pin_setup Statement

Pin_setup Statement

This statement specifies special hardware characteristics of certain pins and determines what pin states can be programmed for those pins. For more information, see [Pin Setups and Valid Character Sets](#).

Syntax

```
pin_setup = {  
    pin-item pin-setup-type ;  
    pin-item pin-setup-type ;  
    ...  
}
```

A pin-item can be:

- A pin name or pin group name (defined in a pin map)
- Any combination of pins and pin groups enclosed in parentheses and separated by commas, in the form:

(item, item, ..., item)

[Pingroups used in a pin_setup](#) statement can be identical to pingroups in a vector block pinlist, or supersets of them.

[A pin cannot appear in multiple pin groups used in a pin_setup](#) statement.

The valid pin setup types are as follows:

2x	Allows two pin state characters per pin in a single vector, an early state and a late state. The 2x setup is intended for pins defined as IO in a Pin
----	---

Map sheet. If applied to an input-only pin, it is treated as the 2x_input setup; if applied to an output-only pin, it is treated as the 2x_output setup.

For valid pin states, see [Pin Setup 2x](#).

2x_input

Allows two drive pin state characters per pin in a single vector, an early state and a late state. The setup can be used only for pins defined as input-only or as I/O in a Pin Map sheet.

For valid pin states, see [Pin Setup 2x_input](#).

2x_output

Allows two receive pin state characters per pin in a single vector, an early state and a late state. The setup can be used only for pins defined as output-only or as I/O in a Pin Map sheet.

For valid pin states, see [Pin Setup 2x_output](#).

2x_agp

Allows two pin state characters per pin in a single vector, an early state and a late state. The setup is used with an Accelerated Graphics Port and provides a dedicated high-bandwidth connection from the memory/CPU to a video card.

For valid pin states, see [Pin Setup 2x_agp](#).

2x_scio

Allows two pin state characters per pin in a single vector, an early and late state. Use this setup with pins defined as I/O in a Pin Map sheet. This mode muxes the data bits of two adjacent channels, which requires proper routing of signals on the DIB.

This mode provides more pin state flexibility than the other 2X modes, including drive and compare in the same cycle.

For valid pin states, see [Pin Setup 2x_scio](#).

2x_scio_ex

Available only in HSDMQ patterns, which use the UltraPin1600 digital instrument. Typically, this mode is only used with quad timing mode patterns, but it is also valid with single and dual timing modes. Allows two pin state characters per pin in a single vector, an early and late state. Use this setup with pins defined as I/O in a Pin Map sheet. This mode muxes the data bits of two adjacent channels, which requires proper routing of signals on the DIB.

This mode provides expanded pin state flexibility beyond that of the 2x_scio mode, allowing two additional pin states in early and late cycles, X0 and X1. These states are interpreted as “don’t care” for the early cycle and “drive to high or low” for the late cycle. This mode is not supported with Digital Signal Source and Capture (DSSC) patterns.

For valid pin states, see Pin Setup 2x_scio_ex.

2x_scio_ex_m

Available only in HSDMQ patterns, which use the UltraPin1600 digital instrument. Typically, this mode is only used with quad timing mode patterns, but it is also valid with single and dual timing modes. Allows two pin state characters per pin in a single vector, an early and late state. Use this setup with pins defined as I/O in a Pin Map sheet. This mode muxes the data bits of two adjacent channels, which requires proper routing of signals on the DIB.

This mode provides expanded pin state flexibility beyond that of the 2x_scio_ex mode, allowing additional pin states using the expect midband pin character (MM, ML, LM, MH, and HM). This mode also supports pin states using the DigCap extended microcode Store (LL/Store, LH/Store, HL/Store, HH/Store, MM/Store, ML/Store, LM/Store, MH/Store, and HM/Store) and Trig (XX).

This mode is not supported with Digital Signal Source (DigSrc) patterns.

For valid pin states, see Pin Setup 2x_scio_ex_m.

window

Defines a window that can be used over an arbitrary number of cycles. For example, you can use this pin setup to open a window and leave it open until a certain device condition is met in a match loop (for example, a particular device pin goes high). You can’t use a predefined window time on the edge set sheet because the device will go high at some point. In this case, you can use a window pin setup to open a window early in the pattern burst and then close it after the match loop completes.

For valid pin states, see Pin Setup Window.

high_voltage

Defines a high-voltage pin.

Available only on the HSDM UltraPin800 digital instrument.
For valid pin states, see Pin Setup high_voltage.

Pin Setups in Pattern Bursts

In a pattern burst, all patterns that use a specific pin must have the same pin setup for that pin.

Example:

```
pin_setup = {  
    pin0    2x;  
    pin1    high_voltage;  
}
```

< >	
	2015 Teradyne, Inc. All rights reserved.

plheaderstatements.2.11

<

>

Pattern Header and Setup Statements : Pin Setups and Valid Character Sets

Pin Setups and Valid Character Sets

The `pin_setup` statement for a pin determines what characters are valid for that pin. It also determines the behavior of certain of the characters because some characters have different meanings in different pin setup contexts.

The following topics describe what characters are valid for each pin setup:

- Default Pin Setup
- Pin Setup 2x_input
- Pin Setup 2x_output
- Pin Setup 2x_agp
- Pin Setup 2x
- Pin Setup 2x_scio
- Pin Setup 2x_scio_ex
- Pin Setup 2x_scio_ex_m
- Pin Setup Window

- [Pin Setup high_voltage](#)

Note:

These pin setup modes do not apply to the UltraPin4000 and HPM instruments. The 2x_scio_ex mode is only supported in hsdmq patterns, which use the UltraPin1600 digital instrument. The section also includes these topics:

- [Pin State Characters and Extended Microcodes](#)

- [Determining Valid Combinations](#)

- [Restrictions on Enhanced Pin Setups](#)

[Pin State Characters and Extended Microcodes](#)
The following are the pin state characters:

Pin State Characters

Character	Description
0	Drive Low
1	Drive High
2	Drive High Voltage
L	Expect Low
H	Expect High
M	Expect Mid-band
V	Expect Valid
X	Mask
W	Window Strobe
D	Drive ADS
I	Drive inverse ADS
E	Expect ADS
C	Expect inverse ADS
-	Repeat previous state

For HSDM (HSD1000 and UltraPin800) and HSDMQ (UltraPin1600), the Digital Signal Capture (DigCap) instrument has its extended microcodes encoded as part of the pin data. You must consider these extended microcodes along with the pin state characters when defining valid character sets for pin setups. The following DigCap extended microcode combinations are allowed in an HSDM or HSDMQ pattern file:

Digital Signal Capture Extended Microcodes

Microcode	Description
Trig	Starts a capture segment.
Store	Stores a data sample
Trig, Store	Combination of Trig and Store
Store, Inst_Cond_Strobe	Executes a Store and gates the internally generated condition signal to be acted on

 Note:

The UltraPin4000 digital instrument does not support these DigCap extended microcode. It supports STV only. HPM does not support DigCap.

Determining Valid Combinations

The valid combinations of pin state characters and DigCap extended microcodes that can be programmed for a pin depend on the following conditions:

- Pin setup of the pin
- Whether the pin is a differential pin and, if so, its differential programming method: complementary or true-differential or diff-cm (common mode)
- If the pin is a differential pin using diff-cm programming method, whether it is a "D" pin (differential voltage) or a "C" pin (common mode voltage)
- Whether the MTO instrument is used in the pattern file
- Whether the pin is a DSSC pin
- If the pin has the 2X pin setup, whether it is an input only, output only, or I/O pin

Restrictions on Enhanced Pin Setups

Certain pin setups are considered enhanced pin setups. These setups require an additional hardware resource, the source select memory, to map the correct data to the DUT. Because of this extra resource, there are restrictions on using enhanced pin setups. The following are the enhanced pin setups:

- Default pin setup with MTO
- Default pin setup on DSSC pins when MTO is not used
- 2x_output pin setup for nondifferential and complementary differential pins
- 2x_output pin setup for differential-common mode pins
- 2x_agp pin setup
- 2x pin setup for nondifferential and complementary differential pins

A scan pattern cannot use any enhanced pin setup configuration. For other patterns, use only one enhanced pin setup in a pattern. If you use more than one, carefully follow the restrictions on what combination of pin data characters you can use. You can use four source select addresses. Each source select address can program eight different pin data states or extended microcodes. Because the total number of pin data states and extended microcodes is greater than eight, this means that each source select address can specify a subset of the total. Here, the combination for each source select address is represented as columns in a table, like this one:

Enhanced Pin Setup for Default with MTO

Source Select Address			
0	1	2	3
0	0	0	0
1	1	1	1
L	L	L	L
X	X	X	X

H	H	H	H
D	C	Store/L	Store/L
E	I	Store/H	Store/H
V	V	Store/M	Store/V

All pins that use an enhanced pin setup in a pattern file must use the same source select address on a single vector. For example, two pins use the enhanced pin setup for default with MTO, as shown in the table above. The first pin uses pin data state D from source select address 0. In the same vector, the second pin cannot use pin data states from source select addresses 1 through 3. So, it cannot use C, I, Store/L, Store/H, Store/M, or Store/V.

If pins use different enhanced pin setups in a pattern file, then the same source select address is used for all extended pin setups in each vector. This means that, on a single vector, all pins with enhanced pin setups must use data only from the same column in their respective tables.

Assume that you have a pattern that uses the enhanced pin setup for default with MTO (table above) for pin1 and the enhanced pin setup for 2x_output (table below) for pin2:

Enhanced Pin Setup for 2x_output

Source Select Address			
0	1	2	3
LL	LL	LL	LL
LH	LH	LH	LH
HL	HL	HL	HL
HH	HH	HH	HH
XX	XX	XX	XX
LX	XL	LX	XL
HX	XH	HX	XH
Store/V V	Store/V V	Store/V V	Store/V V

On a vector, suppose you use pin data state C for pin1. C uses source select address 1 (column 2) of the “Enhanced Pin Setup for Default with MTO” table. For pin2 on the same vector, you can use only values from source select address 1 (column 2) from the “Enhanced Pin Setup for 2x_output” table. As a result, pin2 could use pin data state XL, but not LX, since LX is found on source select addresses 0 and 2.

Note that some values appear in more than one column. For example, suppose you use 0 for pin1. 0 appears in all four source select addresses. This means that you can use any of the values found in any of the source select addresses for pin2. Finally, suppose you use Store/H for pin1. That restricts you to source select addresses 2 and 3 for pin2.

<		>	
	2015 Teradyne, Inc. All rights reserved.		

plheaderstatements.2.12

<

>

Pattern Header and Setup Statements : Pin Setups and Valid Character Sets : Default Pin Setup

Default Pin Setup

If you do not specify the pin setup of a pin in a pin_setup statement, IG-XL will use a default pin setup.

- Default for Nondifferential or Complementary Differential Pins
- Default for True Differential Pins
- Default for Differential-Common Mode Pins
- Default for Nondifferential or Complementary Differential Pins With MTO
- Default for Nondifferential or Complementary Differential Pins With MTO Dash
- Default for Nondifferential or Complementary Differential Pins With DSSC

Default for Nondifferential or Complementary Differential Pins

You can program the following pin states for pins in 1X mode that do not use MTO or DSSC. Data is the same for all source select addresses.

Default 1X Pin Setups	
Pin State	Description
0	Drive Low

1	Drive High
L	Expect Low
H	Expect High
M	Expect Mid-band
V	Expect Valid
X	Mask
-	Repeat previous state

DigCap extended microcodes are not allowed.

Default for True Differential Pins

Program true differential pins using any of these combinations:

- Pin state characters without DigCap extended microcodes: 0, 1, L, H, M, V, X, -, D, I, E, C.
- Combinations of L, H, M, V, X, E, C (receive states) with any DigCap extended microcode combination
- Combinations of 0, 1, and Trig

Default for Differential-Common Mode Pins

Program differential-common mode pins using these pin state characters: 0, 1, L, H, M, V, X, and -.

No DigCap extended microcodes are allowed.

☒ **Note:**

When the differential-common mode programming method is used, the pin state characters on both sides must be either drive or compare. In this case, mixing drive and compare states for a differential pair in a tester cycle is invalid.

Default for Nondifferential or Complementary Differential Pins With MTO

If the MTO is programmed in the instruments statement, you can program using these combinations of pin state characters and DigCap extended microcodes:

Enhanced Pin Setup for Default with MTO

Source Select Address				
0	1	2	3	
0	0	0	0	

1	1	1	1
L	L	L	L
H	H	H	H
X	X	X	X
D	C	Store/L	Store/L
E	I	Store/H	Store/H
V	V	Store/M	Store/V

On reading this table, see Restrictions on Enhanced Pin Setups.

Default for Nondifferential or Complementary Differential Pins With MTO Dash

If the MTO is programmed in the instruments statement, and if the compiler control option allow_mto_dash is set to yes, you can program using these combinations of pin state characters and DigCap extended microcodes:

Enhanced Pin Setup for Default with Allow MTO Dash

Source Select Address			
0	1	2	3
0	0	0	0
1	1	1	1
L	L	L	L
H	H	H	H
D	D	D	D
X	X	X	X
-	-	-	-
E	C	I	V

On reading this table, see Restrictions on Enhanced Pin Setups.

Default for Nondifferential or Complementary Differential Pins With DSSC

You can program using these combinations of pin state characters and DigCap extended microcodes when the pin is a DSSC pin:

Enhanced Pin Setup for Default without MTO

Source Select Address			
0	1	2	3
0	0	0	0
1	1	1	1
L	L	L	L
H	H	H	H
X	X	X	X
Store/X	Store/X	V	Trig, Store/V
Store/L	Store/V	Store/V	Store/V
Store/H	D	D	Trig/V

On reading this table, see Restrictions on Enhanced Pin Setups.

<

>

2015 Teradyne, Inc. All rights reserved.

plheaderstatements.2.13

<	>
---	---

Pattern Header and Setup Statements : Pin Setups and Valid Character Sets : Pin Setup 2x_input

Pin Setup 2x_input

This pin setup can be used only on pins that are defined as input-only or I/O in a Pin Map sheet. It allows two pin state characters per pin in a single vector, that is, in a tester cycle: an early state and a late state.

Note:

This is not an enhanced pin setup mode because it does not use any source select address bits.

Nondifferential or Complementary Differential Pins

The available pin state combinations are: 00, 01, 10, 11, 0X, 1X, and XX. The first character is the early state. The second character is the late state. DigCap extended microcodes are not allowed.

True Differential Pins

This pin setup does not support true differential programming method.

Differential-Common Mode Pins

For a "D" pin, the available pin state combinations are 00, 01, 10 and 11.

For a "C" pin, the available pin state combinations are 00 and 11. This means that changing common mode voltage in the middle of a tester cycle is not supported.

Note:

When the differential-common mode programming method is used, the pin state characters on both sides must be either drive or compare. In this case, mixing drive and compare states for a differential pair in a tester cycle is invalid.

<	>
---	---

plheaderstatements.2.14

<

>

Pattern Header and Setup Statements : Pin Setups and Valid Character Sets : Pin Setup 2x_output

Pin Setup 2x_output

This pin setup can be used only on pins that are defined as output-only or I/O in a Pin Map sheet. It allows two pin state characters per pin in a single vector, that is, in a tester cycle: an early state and a late state.

Nondifferential or Complementary Differential Pins

You can program using these combinations of pin state character and DigCap extended microcodes:

Enhanced Pin Setup for 2x_output

Source Select Address			
0	1	2	3
LL	LL	LL	LL
LH	LH	LH	LH
HL	HL	HL	HL
HH	HH	HH	HH
XX	XX	XX	XX
LX	XL	LX	XL
HX	XH	HX	XH
Store/V V	Store/V V	Store/V V	Store/V V

On reading this table, see Restrictions on Enhanced Pin Setups.

True Differential Pins

This pin setup does not support true differential programming method.

Differential-Common Mode Pins

For a “D” pin, you can program using these combinations of pin state characters. DigCap extended microcodes are not allowed:

Enhanced Pin Setup for D Pin

Source Select Address			
0	1	2	3
LL	LL	LL	LL
LH	LH	LH	LH
HL	HL	HL	HL
HH	HH	HH	HH
XX	XX	XX	XX
LX	XL	LX	XL
HX	XH	HX	XH
VV	VV	VV	VV

For a “C” pin, you can program using these combinations of pin state characters. DigCap extended microcodes are not allowed:

Enhanced Pin Setup for C Pin

Source Select Address			
0	1	2	3
LL	LL	LL	LL
LH	LH	LH	LH
HL	HL	HL	HL
HH	HH	HH	HH
XX	XX	XX	XX
LX	XL	LX	XL
HX	XH	HX	XH
MM	MM	MM	MM

On reading these tables, see [Restrictions on Enhanced Pin Setups](#).

☑ Note:

When the differential-common mode programming method is used, the pin state characters on both sides must be either drive or compare. In this case, mixing drive and compare states for a differential pair in a tester cycle is invalid.

< >		
	2015 Teradyne, Inc. All rights reserved.	

plheaderstatements.2.15

<

>

Pattern Header and Setup Statements : Pin Setups and Valid Character Sets : Pin Setup 2x_agp

Pin Setup 2x_agp

This pin setup allows two pin state characters per pin in a single vector, that is, in a tester cycle: an early state and a late state.

Nondifferential or Complementary Differential Pins

You can program using these combinations of pin state character and DigCap extended microcodes:

Enhanced Pin Setup for 2x_agp

Source Select Address			
0	1	2	3
00	00	00	00
01	01	01	01
10	10	10	10
11	11	11	11
XX	XX	XX	XX
LX	XL	LX	XL
HX	XH	HX	XH
Store/V V	Store/V V	Store/V V	Store/V V

On reading this table, see Restrictions on Enhanced Pin Setups.

True Differential Pins

This pin setup does not support true differential programming method.

Differential-Common Mode Pins

This pin setup does not support differential-common mode programming method.

<		>	
	2015 Teradyne, Inc. All rights reserved.		

plheaderstatements.2.16

<

>

Pattern Header and Setup Statements : Pin Setups and Valid Character Sets : Pin Setup 2x

Pin Setup 2x

If you program this pin setup for an input-only pin, the pin is considered as having a 2x_input pin setup instead. Likewise, if you program this pin setup for an output-only pin, the pin is considered as having a 2x_output pin setup. Therefore, this section applies only to pins that are defined as I/O in a Pin Map sheet. It allows two pin state characters per pin in a single vector, that is, in a tester cycle: an early state and a late state.

Nondifferential or Complementary Differential Pins

You can program using these combinations of pin state character and DigCap extended microcodes:

Enhanced Pin Setup for 2x

Source Select Address			
0	1	2	3
00	00	LL	LL
01	01	LH	LH
10	10	HL	HL
11	11	HH	HH
XX	XX	XX	XX
0X	0X	LX	XL
1X	1X	HX	XH
Store/V V	Store/V V	Store/V V	Store/V V

On reading this table, see Restrictions on Enhanced Pin Setups.

True Differential Pins

You can program using these combinations:

- | | |
|---|---|
| • | Any early/late drive combination from 0 and 1 |
| • | Any early/late compare combination from L, H and X |
| • | Any early/late compare combination from M and X |
| • | Any early/late compare combination from V and X |
| • | 0X, 1X, DD, II, EE, CC |
| • | Any early/late compare combination from L and H, with DigCap extended microcode Store |
| • | VV or XX with DigCap extended microcode Store |
| • | VV or XX with DigCap extended microcode Trig |
| • | VV or XX with DigCap extended microcode combination Trig, Store |
| • | VV/Store, Inst_Cond_Strobe |

Differential-Common Mode Pins

This pin setup does not support differential-common mode programming method.

<

>

2015 Teradyne, Inc. All rights reserved.

plheaderstatements.2.17

<	>
---	---

Pattern Header and Setup Statements : Pin Setups and Valid Character Sets : Pin Setup 2x_scio

Pin Setup 2x_scio

This mode allows two pin state characters per pin in a single vector, that is, in a tester cycle: an early state and a late state. This mode applies only to pins that are defined as I/O in a Pin Map sheet.

☒ **Note:**

This is not an enhanced pin setup mode because it does not use any source select address bits. SCIO muxes data from two adjacent channels for a given pin resulting in six bits of pin data and more pin state flexibility, allowing drive and compare in the same cycle. IG-XL checks that the adjacent channels are not used by other pins.

For a pattern file example that uses this mode, see [Data Mux 2X SCIO Pin Mode Pattern File](#). This mode muxes the data bits of two adjacent channels, which requires proper routing of signals on the DIB. For DIB channel selection guidelines for this mode, see [Data Mux 2X SCIO Mode](#).

Nondifferential, Complementary Differential, and True Differential Pins

You can program using these combinations:

- | | |
|---|---|
| • | Any early/late drive combination from 0 and 1 |
| • | Any early/late compare combination from L, H and X |
| • | Early drive and late compare combinations: 0L, 0H, 0X, 1L, 1H, 1X |
| • | Any early/late compare combination from M and X |
| • | Any early/late compare combination from V and X |

- [ADS combinations: DD, II, EE, CC \(For DigSrc\)](#)
- [Any early/late combination from L and H, with DigCap extended microcode Store](#)
- [VV with DigCap extended microcode Store](#)
- [VV or XX with DigCap extended microcode Trig](#)
- [VV with DigCap extended microcode combination Trig, Store](#)
- [VV/Store, Inst_Cond_Strobe](#)
- [All of pin state combinations above with "STV"](#)

The following pin state combinations are not supported:

- [XX with DigCap extended microcode Store](#)
- [XX with DigCap extended microcode combination Trig, Store](#)
- [Any early compare and late drive combinations: L0, H0, X0, L1, H1, X1](#)

Differential-Common Mode Pins

This pin setup does not support differential-common mode programming method.

plheaderstatements.2.18

<	>
---	---

Pattern Header and Setup Statements : Pin Setups and Valid Character Sets : Pin Setup 2x_scio_ex

Pin Setup 2x_scio_ex

Like the 2x_scio mode, this mode allows two pin state characters per pin in a single vector, that is, in a tester cycle: an early state and a late state. This mode provides expanded pin state flexibility, however, beyond that of the 2x_scio mode, allowing two additional pin states in early and late cycles, X0 and X1. These states are interpreted as "don't care" for the early cycle and "drive to high or low" for the late cycle.

Note:

This is not an enhanced pin setup mode because it does not use any source select address bits. This mode applies only to pins that are defined as I/O in a Pin Map sheet. It is only supported in hsdmq patterns, which use the UltraPin1600 digital instrument. This mode is not supported with Digital Signal Source and Capture (DSSC) patterns.

SCIO muxes data from two adjacent channels for a given pin resulting in six bits of pin data and more pin state flexibility, allowing drive and compare in the same cycle. IG-XL checks that the adjacent channels are not used by other pins.

This mode muxes the data bits of two adjacent channels, which requires proper routing of signals on the DIB. For channel selection guidelines when using this mode on an UltraPin1600 DIB, see Data Mux 2X SCIO Modes.

For a pattern file example that uses this mode, see Quad Data Mux 2X SCIO Expanded Pin Mode Pattern File.

Nondifferential, Complementary Differential, and True Differential Pins

You can program using these combinations:

- | | |
|---|--|
| • | Any early/late drive combination from 0 and 1 |
| • | Any early/late compare combination from L, H and X |

- Any early/late compare combination from M and X
- Any early/late compare combination from V and X
- Early compare and late drive combinations: X0, X1
- Early drive and late compare combinations: 0L, 0H, 0X, 1L, 1H, 1X

The following pin state combinations are not supported:

- ADS combinations: DD, II, EE, CC (For DigSrc)
- Any early/late combination from L and H, with DigCap extended microcode Store
- VV with DigCap extended microcode Store
- VV or XX with DigCap extended microcode Trig
- VV with DigCap extended microcode combination Trig, Store
- VV/Store, Inst_Cond_Strobe
- XX with DigCap extended microcode Store
- XX with DigCap extended microcode combination Trig, Store
- Early compare and late drive combinations: L0, H0, L1, H1
- All of pin state combinations above with "STV"

Differential-Common Mode Pins

This pin setup does not support differential-common mode programming method.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

plheaderstatements.2.19

<	>
---	---

Pattern Header and Setup Statements : Pin Setups and Valid Character Sets : Pin Setup 2x_scio_ex_m

Pin Setup 2x_scio_ex_m

Like the 2x_scio_ex mode, this mode allows two pin state characters per pin in a single vector, that is, in a tester cycle: an early state and a late state. This mode supports a different set of pin state combinations from that of the 2x_scio_ex mode. This mode provides combinations using the expect midband pin state (MM, ML, LM, MH, and HM). This mode also supports pin states using the DigCap extended microcode Store (LL/Store, LH/Store, HL/Store, HH/Store, MM/Store, ML/Store, LM/Store, MH/Store, and HM/Store) and Trig (XX). Finally, it removes support for the expect valid (V) pin state.

Note:

This is not an enhanced pin setup mode because it does not use any source select address bits. This mode applies only to pins that are defined as I/O in a Pin Map sheet. It is only supported in hsdmq patterns, which use the UltraPin1600 digital instrument. This mode is not supported with Digital Signal Source (DigSrc) patterns.

SCIO muxes data from two adjacent channels for a given pin resulting in six bits of pin data and more pin state flexibility, allowing drive and compare in the same cycle. IG-XL checks that the adjacent channels are not used by other pins.

This mode muxes the data bits of two adjacent channels, which requires proper routing of signals on the DIB. For channel selection guidelines when using this mode on an UltraPin1600 DIB, see Data Mux 2X SCIO Modes.

For a pattern file example that uses this mode, see Quad Data Mux 2X SCIO Expanded Midband Pin Mode Pattern File.

Nondifferential, Complementary Differential, and True Differential Pins

You can program using these combinations:

- Any early/late drive combination from 0 and 1: 00, 01, 10, 11

- Any early/late compare combination from L, H, X, and M: LL, LH, HL, HH, LX, XL, HX, XH, XX, MM, MX, XM, ML, LM, MH, HM
- Early drive and late compare combinations: 0L, 0H, 0X, 1L, 1H, 1X, 0M, 1M
- Early compare and late drive combinations: X0, X1
- Any early/late compare combination from L, H, and M with DigCap extended microcode Store: LL/Store, LH/Store, HL/Store, HH/Store, MM/Store, ML/Store, LM/Store, MH/Store, HM/Store
- XX with DigCap extended microcode Trig

The following pin state combinations are not supported:

- ADS combinations: DD, II, EE, CC (For DigSrc)
- Any early/late compare combination (with or without DigCap) that uses V
- XX with DigCap extended microcode Store
- XX with DigCap extended microcode combination Trig, Store
- Early compare and late drive combinations: L0, H0, L1, H1
- All of pin state combinations above with "STV"

Differential-Common Mode Pins

This pin setup does not support differential-common mode programming method.

<

>

2015 Teradyne, Inc. All rights reserved.

plheaderstatements.2.20

<	>
-----------------	-----------------

Pattern Header and Setup Statements : Pin Setups and Valid Character Sets : Pin Setup Window

Pin Setup Window

Regardless of the differential programming method, you can use these pin state characters to program pins that have the “window” pin setup: 0, 1, L, H, M, V, X, and W. DigCap extended microcodes are not allowed.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

plheaderstatements.2.21

<	>
-----------------	-----------------

Pattern Header and Setup Statements : Pin Setups and Valid Character Sets : Pin Setup high_voltage

Pin Setup high_voltage

This pin setup can be used for nondifferential pins only. It is only supported on the UltraPin800 digital instrument (hsdm). You can use the following pin state characters: 0, 1, 2, and -. DigCap extended microcodes are not allowed.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

plheaderstatements.2.22



Pattern Header and Setup Statements : Instruments Statement

Instruments Statement

The instruments statement defines instruments that have extended microcode programmed in a pattern. For per-pin instruments, the statement also associates a pin with the instrument, information that is used when the pattern file is compiled. The statement may also take addition parameters for specific instruments.

Information on Instrument-Specific Syntax

Each instrument has its own syntax for the instruments statement.

For the instruments statement syntax for specific instruments, click Related Topics.

The remainder of this topic is a general discussion of the instruments statement, containing points applicable to all instruments.

Syntax

The syntax of the instruments statement depends on the instrument itself. The general format for this statement for per-pin instruments is:

```
instruments = {
    pin-item:instrument-type parameters ;
    ...
}
```

For nonpin instruments, the general format is:

```
instruments = {
    instrument-type parameters ;
    ...
}
```

In each case, the presence of parameters depends on the instrument requirements. Some instruments do not take parameters.

Requirement for Instruments in Pattern Bursts

If one pattern module in a burst uses instrument microcode, then all patterns in the burst must have the same instrument setup, even if the other pattern modules do not use instrument microcode. A pattern burst is executed as a single unit, with only one pattern start. Therefore, the instrument can be set up only once, at the start of the burst. For this reason, all pattern modules in a burst must have the same instrument setup.

The best solution is to put identical instruments statements in all pattern files used in a burst if one pattern file has an instruments statement. This solution does not have a performance penalty.

<		>
	2015 Teradyne, Inc. All rights reserved.	

plheaderstatements.2.23

<	>
-----------------	-----------------

Pattern Header and Setup Statements : Scan_pins Statement

Scan_pins Statement

The scan_pins statement is required for pattern files that have scan data.
This statement is not allowed in quad timing mode (opcode_mode = quad), which is used in hsdmq patterns on the UltraPin1600.
For details, see scan_pins in the SCAN section.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

plmicrocode.4.01

<	>
-----------------	-----------------

Microcode (Opcodes and Control Bits)

Microcode (Opcodes and Control Bits)

This section describes the microcode portion of the vector data.
The following topics are available:

- [Opcodes and Control Bits Overview](#)
- [List of Opcodes and Control Bits](#)
- [Loop Opcodes](#)
- [Repeat Opcodes](#)
- [Branch Opcodes](#)
- [Subroutine Opcodes](#)
- [Counter Opcodes](#)
- [CPU Flag Condition Opcodes](#)
- [Miscellaneous Opcodes](#)
- [Control Bits](#)

- [Conditional Statements](#)
- [Instrument Microcode](#)
- [Microcode Groups](#)

<

>

plmicrocode.4.02

<

>

Microcode (Opcodes and Control Bits) : Opcodes and Control Bits Overview

Opcodes and Control Bits Overview

This topic includes the following items:

- Definitions
- Descriptions of Opcodes and Control Bits
- Rules and Notes for Opcodes and Control Bits

Definitions

This section defines the opcodes and control bits that are recognized by the pattern compiler.

opcode	Controls the flow of execution within the pattern file: for example, loop on this vector, or jump to a specific label.
control bit	Modifies execution of the specific vector on which it appears: for example, mask failure on this vector, or store the vector in HRAM.

Descriptions of Opcodes and Control Bits

For a list of opcodes and control bits, see [List of Opcodes and Control Bits](#). From there, links are available to descriptions of the opcodes and control bits, grouped by category.

Rules and Notes for Opcodes and Control Bits

Location in vector	Opcodes and control bits must appear after any label and before the > (greater than) character that separates the opcode field from the channel
--------------------	---

data.

Number per vector	With few exceptions, you can place only one opcode in a single vector. You can place any number of control bits in a single vector, which can also contain an opcode. Use white space or commas to separate the control bits and opcode.
Opcode groups	A group is a unique combination of microcodes in a vector. A test program can include up to 249 microcode groups per pattern start per time domain. See Microcode Groups.
Opcode modes	An opcode can appear only on the final vector of a vector group. In each mode, this means: • In single and By1 mode, on any vector. • In dual mode, only on the second vector in a two-vector group. • In quad mode, only on the fourth vector of a four-vector group. • In By16 or By16lIO mode, only on the 16th vector of a 16-vector group.
VM and SRM	For the HSDM (HSD1000 and UltraPin800), HSDMQ (UltraPin1600), and HSD4G (UltraPin4000 and HPM) digital instruments, pattern modules are explicitly assigned to Vector Memory (VM) or Subroutine Memory (SRM). Most opcodes can be used in either VM or SRM. A few can be used only in SRM. These opcodes are clearly noted. See Pattern Modules.

plmicrocode.4.03

<	>
-----------------	-----------------

Microcode (Opcodes and Control Bits) : List of Opcodes and Control Bits

List of Opcodes and Control Bits

This topic includes the following items:

- | | |
|---|-------------------------------------|
| • | Categories |
| • | Information on Instrument Microcode |
| • | Notes on Descriptions |

Categories

Opcodes and control bits can be grouped into the following categories:

Loop Opcodes

- loop
- end_loop
- push_loop
- pop_loop
- clr_loop

- [wflag](#)

Repeat Opcodes

- [repeat](#)
- [pipe_minus](#)

Branch Opcodes

- [halt](#)
- [branch_expr](#)
- [jump](#)
- [match](#)
- [match_vm](#)
- [jump_vm](#)

Subroutine Opcodes

- [call](#)
- [resume](#)
- [return](#)
- [refresh_subr](#)
- [push_subr](#)
- [pop_subr](#)
- [clr_subr](#)

- [stop_subr](#)

Counter Opcodes

- [set_msb](#)
- [set_msb_infinite](#)
- [set Counter](#)
- [set_infinite](#)
- [reload](#)
- [reload_all_ctr](#)
- [clr_sync_ctr](#)
- [enable_sync_ctr](#)
- [disable_sync_ctr](#)
- [enable_fail_count](#)
- [disable_fail_count](#)
- [refresh_w2](#)
- [refresh_w1](#)
- [refresh_w0](#)

CPU Flag Condition Opcodes

- [set_cpu_cond](#)

	• clr_cond_flags
	• inv_cond_flags
Miscellaneous Opcodes	• send_patgen_cond • walking_strobe_start (UltraPin1600 only) • nop
Control Bits	• care • stv • mask • refresh_en

Information on Instrument Microcode

Each supported instrument has its own set of microcodes. See the instrument sections for the microcode that each instrument supports.

See also the general description of Instrument Microcode.

Notes on Descriptions

The description of each opcode indicates these points:

- | | |
|---|--|
| • | Whether the opcode takes an operand: a label or relative address, or a number in the specified range. |
| • | Whether the opcode can be made conditional in an if statement. Unless explicitly noted, an opcode cannot be made conditional. See Conditional Statements . |

- Whether the opcode can be used in VM and SRM or only in an SRM module. Unless explicitly noted, the opcode can be used in either VM or SRM. See [Pattern Modules](#).

< >	
	2015 Teradyne, Inc. All rights reserved.

plmicrocode.4.04

<

>

Microcode (Opcodes and Control Bits) : Loop Opcodes

Loop Opcodes

Loop Opcodes

Opcode	Description
loop counter	Specifies a counter to be used for a loop. Its operand specifies the loop counter. The ID of counter is pushed onto counter stack. The pattern uses the value to execute an SRM loop. Typically, you place this opcode before and outside the loop that is repeated. Do not place it inside the loop; otherwise, you may encounter a PatGen counter stack overflow error. You can use this opcode only in SRM, and the loop must use an end_loop opcode. In single mode, you can place the loop counter on the same vector as the SRM label (that is, the target vector specified by the end_loop opcode). In dual mode, you can place the loop counter within the same vector pair as the SRM label (that is, the target vector specified by the end_loop opcode). In quad mode, you can place the loop counter within the same group of four vectors as the SRM label (that is, the target vector specified by the end_loop opcode). In each of these cases (single, dual, and quad), the loop opcode becomes a NOP on the second and subsequent executions resulting from the loop.

end_loop identifier

Specifies an SRM label that is the conditional end of a loop. The opcode checks the value of the counter whose ID is on top of counter stack:

- If the counter reaches 0, pop the top of the counter stack, and then execute the next vector.
- Otherwise, decrement the counter, and then jump to the SRM label specified by the identifier.

You can use this opcode with SRM only, and must be used with a loop opcode. The operand can also be a relative address.

The end_loop opcode uses a counter that has been previously pushed on top of the counter stack by a push_loop or loop opcode. It is a hardware restriction that such counter must be loaded (by the set_msb and set opcodes) two opcode cycles before being used by the end_loop opcode.

The pattern compiler cannot enforce this restriction, because it cannot track the pattern flow when looping and branching are used in the pattern.

You cannot use this opcode with the modifier refresh_en on the same vector.

push_loop counter

Pushes the counter ID onto the counter stack.

pop_loop

Pops the top item on the counter stack.

clr_loop

Removes all items on the counter stack.

wflag

Makes pattern execution wait on the condition set by the latest branch_expr opcode, with a time-out. The counter whose ID is on top of the counter stack controls the time-out.

The opcode executes as follows:

- At each tester cycle, if the branch_expr condition is true, the next vector is executed.
- Otherwise, if the counter has reached zero, the counter ID is popped off the stack, and the next vector is executed.

- Otherwise, if the `branch_expr` condition is not true, and the counter has not reached zero, the counter is decremented, and the vector is executed again in the next tester cycle.

This opcode can be used to wait indefinitely on a flag condition.
For example:

```
branch_expr=...    // Set flag condition
set_msb_infinite   // Set c1 to infinite value
set_infinite c1
push_loop c1
wflag              // Loop indefinitely on this vector
                  // until condition is satisfied
```

You cannot use this opcode with the modifier `refresh_en` in a VM vector.

plmicrocode.4.05

<

>

Microcode (Opcodes and Control Bits) : Repeat Opcodes

Repeat Opcodes

Repeat Opcodes

Opcode	Description
repeat integer	Causes the vector to be executed integer times, where the operand integer is a decimal number from 1 to 65536. The opcode cannot be used with the modifier refresh_en in a VM vector.
repeat counter	Causes the vector to be executed K times, if the counter is loaded using these opcodes: set_msb K set counter K If the counter is loaded using a VBT command, load it with a value of K - 1. On HSD1000, UltraPin800, UltraPin1600, UltraPin4000, and HPM, you can use 16 registers as loop counters, repeat counters, or simply user registers. When accessed through the pattern (opcode) interface, IG-XL assumes you are using it as a loop or repeat counter. In this case, the value that is loaded into the register is one less than what is specified because IG-XL uses the value to determine how many times to repeat the vector or loop. Effectively, IG-XL executes the vector or loop once and then repeats as many times as the register specifies. By subtracting 1 when loading the register, you get a total execution count that equals what you originally specified.

Note that when using VBT to read and write the registers, IG-XL assumes you are using the registers as registers and not as loop or repeat counters. In this case, IG-XL does not modify the value. You cannot use this opcode with the modifier refresh_en in a VM vector.

pipe_minus integer

Similar to repeat, but repeats the vector for the depth of the fail pipeline minus the specified integer.
The minimum value, 0, effectively specifies a repeat equal to the pipeline depth.
The maximum value that can be specified for integer is pipeline-1. Use this opcode when portability to future test systems with a different fail pipeline depth is a concern.
You cannot use this opcode with the modifier refresh_en in a VM vector.

plmicrocode.4.06

<

>

Microcode (Opcodes and Control Bits) : Branch Opcodes

Branch Opcodes

Branch Opcodes

Opcode	Description
halt	Halts the pattern burst.
branch_expr=(expr)	Enables the specified condition expr for later use in an if (branch_expr) statement. See Conditional Statements.
jump identifier	Transfers execution flow to SRM at the location indicated by the identifier, which is a label in SRM. In SRM, the operand can also be relative address. This opcode can be conditionalized in VM or SRM. You cannot use this opcode in an if (branch_expr) with the modifier refresh_en in the same vector.
match identifier	Makes pattern execution wait on the condition set by the latest branch_expr opcode with a time-out. The time-out is controlled by the counter whose identifier is on top of the counter stack. At each tester cycle, do the following: - If the condition is true, jump to the SRM label specified by the identifier operand. - Otherwise, if the counter reaches zero, pop the counter identifier off the counter stack, and execute the following vector.

- If the counter is not zero, decrement it, and execute this vector again in the next tester cycle.

In SRM, the operand can also be a relative address.

See Match Loop Programming.

You cannot use this opcode with the modifier refresh_en in a VM vector.

match_vm

Similar to the match opcode, except that when the condition is true, it will execute the next VM vector in the pipeline.

See Match Loop Programming.

SRM only.

jump_vm

Transfers execution to the next VM vector in the pipeline.

SRM only.

This opcode can be conditionalized (in SRM).

< >	
	2015 Teradyne, Inc. All rights reserved.

plmicrocode.4.07

<

>

Microcode (Opcodes and Control Bits) : Subroutine Opcodes

Subroutine Opcodes

Subroutine Opcodes

Opcode	Description
call identifier	Calls the subroutine identifier, which is a subroutine label in SRM. In SRM, the operand can also be relative address. The call opcode can be conditionalized in VM or SRM. You cannot use this opcode with the modifier refresh_en on the same vector.
resume	Exchanges the top of the subroutine stack and the address of the next vector. In other words, it pops the subroutine stack, pushes the address of the next vector, and executes the vector at the popped address.
return	Returns from a subroutine. The opcode transfers execution to the address on top of the subroutine stack, and pops it off the stack. You can use this opcode with SRM only. You cannot use this opcode with the modifier refresh_en on the same vector.
refresh_subr identifier	Stores the refresh subroutine address in the hardware. The operand identifier is the label that starts a refresh subroutine. In SRM, the operand can also be a relative address. Note that you must set the refresh subroutine address for each pattern burst. IG-XL does not reload the hardware register containing the refresh subroutine address from a previous pattern burst if the

	refresh pattern is loaded in a different location when starting the next burst.
push_subr	Pushes the address of the following vector onto the subroutine stack. You cannot use this opcode with the modifier refresh_en on the same vector.
pop_subr	Pops the top item on the subroutine stack. The opcode does not change the current vector address. You cannot use this opcode with the modifier refresh_en on the same vector.
clr_subr	Removes all items on the subroutine stack. You cannot use this opcode with the modifier refresh_en on the same vector.
stop_subr identifier	Stores the stop subroutine address in the hardware. The operand identifier is the label that starts a stop subroutine. In SRM, the operand can also be a relative address. See Stop Subroutines.

For more information, see Subroutines.

< >		
	2015 Teradyne, Inc. All rights reserved.	

plmicrocode.4.08

<

>

Microcode (Opcodes and Control Bits) : Counter Opcodes

Counter Opcodes

Counter Opcodes	
Opcode	Description
set_msb integer	Stores the 12 MSBs of (integer-1) in a temporary register, which will be used to set the 12 MSBs of any user counter. The operand integer is a decimal number from 1 to (2²⁸)-1,that is, in the range (1..268435455). Though the 16 LSBs of integer are unused, the pattern compiler stores them in the pattern file so that reverse compilation can get back the original operand. You must use set_msb at least once before using set, even if you are setting the counter to a value < 65336. If the refresh subroutine uses set_msb, do not use the opcode modifier refresh_en with this opcode. See set and set_msb Counter Examples.
set_msb_infinite	Sets all 12 bits of a temporary register to 1s. This is the same temporary register described for set_msb. The effect is to set the register to 0xFFFF. Use this opcode with set_infinite to set a counter to an infinite value (0xFFFFFFFF), to create an infinite loop or an infinite time-out for match, match_vm, or wflag. If the refresh subroutine uses set_msb, do not use the opcode modifier refresh_en with this opcode.

set counter integer

Sets the 12 MSBs of counter to the value in the temporary MSB register set by set_msb and sets the 16 LSBs of counter to the 16 LSBs of (integer-1). The operand integer is a decimal number from 1 to $(2^{28})-1$, that is, in the range (1..268435455). Though the 12 MSBs of integer are not directly used, the pattern compiler stores them in the pattern file so that reverse compilation can get back the original operand.

The following opcode sequence loads COUNTER with (N-1):

```
set_msb N
```

```
set COUNTER N
```

Note these points:

- If you are setting the counter to a value < 65536, you must precede set COUNTER with a set_msb only if you have previously set MSB to a value >= 65536.
- You must use set_msb at least once before using set COUNTER, even if you are initially setting the counter to a value < 65536.

The VBT property TheHdw.Digital.Patgen.Counter can also set the value of a counter. This property modifies all 28 bits of the counter you select to modify, and it sets the counter differently from the set opcode.

The set opcode sets the counter to value-1, so the counter counts down from value-1 to 0 for a total of value counts. For example, set c0 10 sets the c0 counter to 9, so the counter will count down from 9 to 0 in 10 counts.

In contrast, the Counter VBT property sets the counter to the specified value, so the counter counts down from value to 0 for a total of value+1 counts. For example, if the Counter VBT property specifies 10 as the value, the counter is set to 10, so the counter will count down from 10 to 0 for a total of 11 counts. See TheHdw.Digital.Patgen.Counter in the VBT Syntax Reference.

See also set and set_msb Counter Examples.

set_infinite counter

Sets the 12 MSBs of counter to the value in the temporary register set by set_msb_infinite and sets the 16 LSBs of

counter to 1s.

Use this opcode with `set_msb_infinite` to set a counter to an infinite value (0xFFFFFFFF), to create an infinite loop or an infinite time-out for `match`, `match_vm`, or `wflag`.

The following opcode sequence loads `COUNTER` with all 1s (0xFFFFFFFF):

`set_msb_infinite`

`set_infinite COUNTER`

`reload counter counter ...`

Causes the original values of the counters, which are stored in mirrored registers of the counters, to be loaded back into the counters. The operand is a list of counters.

`reload_all_ctr`

Reloads all counters.

`clr_sync_ctr`

Resets the sync counter.

`enable_sync_ctr`

Enables the sync counter.

`disable_sync_ctr`

Disables the sync counter.

`enable_fail_count`

Enables all fail counters, including:

- Module cycle counter
- Absolute cycle counter
- Time counter

This opcode can be used when exiting keepalive mode.

`disable_fail_count`

Disables all fail counters.

This opcode can be used when entering keepalive mode.

`refresh_w2 integer`

Writes the operand into a temporary register, which is dedicated to refresh counter. Later the operand value can be written to the eight MSBs of the refresh counter.

The operand is an 8-bit decimal number (0...255).

Use this opcode with `refresh_w1` and `refresh_w0` to set the refresh counter.

`refresh_w1 integer`

Writes the operand into a temporary register, which is dedicated to refresh counter. Later the operand value can be written to the 10 middle bits of the refresh counter.

refresh_w0 integer

The operand is an 10-bit decimal number (0...1023).

Sets the operand to 10 LSBs of refresh counter and loads its 18 MSBs from a temporary register that has been set by refresh_w1 and refresh_w2.

The operand is an 10-bit decimal number (0...1023).

<		>	
		2015 Teradyne, Inc. All rights reserved.	

plmicrocode.4.09

<	>
---	---

Microcode (Opcodes and Control Bits) : Counter Opcodes : set and set_msb Counter Examples

set and set_msb Counter Examples

Follow these examples when using set and set_msb counters.

- When the set counter is greater than 65536 (2^{16}) and changed to another value that is also greater than 65536, then make sure the set_msb is equal to the set counter value. For example:

```
// As the set counter value is greater than 65536,  
// use the same value for set_msb.  
set_msb 70000  
set C0 70000  
// As the set counter value is changed and is still greater than 65536,  
// continue to use the same value for set_msb.  
set_msb 80000  
set C1 80000
```

- If the set counter value is changed from greater than 65536 to less than 65536, then make sure to set the set_msb value to less than 65536 and equal to the set counter value. For example:

```
set_msb 75000  
set C0 75000  
// As the set counter value is changed from greater than 65536 to less than  
// 65536, set the set_msb value equal to the set counter value.  
set_msb 7500  
set C1 7500
```

- If the set counter value is less than 65536 and changed to another value which is also less than 65536, then you do not need to change the set_msb so that it is equal to the set counter value. For example:

```
set_msb 600
```

```
set C0 600
```

```
// You do not need to change the set_msb value equal to set counter value
```

```
// as long as the set counter value change is still less than 65536.
```

```
set C1 800
```

- If the set counter value is changed from less than 65536 to greater than 65536, then make sure to change the set_msb value to greater than 65536 and equal to the set counter value. For example:

```
set_msb 100
```

```
set C0 100
```

```
// As the set counter value is changed from less than 65536 to greater than
```

```
// 65536, change set_msb so that it is equal to the set counter value.
```

```
set_msb 85000
```

```
set C1 85000
```

<

>

plmicrocode.4.10

<

>

Microcode (Opcodes and Control Bits) : CPU Flag Condition Opcodes

CPU Flag Condition Opcodes

A vector can have only one opcode, except for the opcodes `set_cpu_cond`, `clr_cond_flags`, and `inv_cond_flags`, which can be together in the same vector. If the user flags listed in the opcodes overlap, the action of the last opcode overrides the actions of previous opcodes on those flags. For more information, see [CPU Flags](#).

CPU Flag Condition Opcodes	
Opcode	Description
<code>set_cpu_cond</code> (cpu-conds)	Sets the specified condition <code>cpu-conds</code> , which must be one or more of <code>cpuA_cond</code> , <code>cpuB_cond</code> , <code>cpuC_cond</code> , and <code>cpuD_cond</code> . The conditions are separated by white space. Unmentioned flags are not affected by this opcode. The flags can be used to communicate with the test program, which can both read and set the signals.
<code>clr_cond_flags</code> (cpu-conds)	Clears the user CPU flags in <code>cpu-conds</code> and holds the others. On <code>cpu-conds</code> , see <code>set_cpu_cond</code> .
<code>inv_cond_flags</code> (cpu-conds)	Inverts the user CPU flags in <code>cpu-conds</code> and holds the others. On <code>cpu-conds</code> , see <code>set_cpu_cond</code> .

<

>

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

plmicrocode.4.11

<

>

Microcode (Opcodes and Control Bits) : Miscellaneous Opcodes

Miscellaneous Opcodes

Miscellaneous Opcodes

Opcode	Description
send_patgen_cond	Sends an XCOND message along the ISL bus, which will set the ISL cond flags of other boards in the test system. This opcode can be programmed together with another PatGen opcode in the same vector. This opcode is not allowed in a vector with nondefault extended microcodes of ISL instruments, such as VSM or BBAC.
nop	Executes the vector once and increments the program counter. It is equivalent to a blank opcode.
walking_strobe_start	Starts the UltraPin1600 walking strobe hardware in an HSDMQ pattern. Use this opcode to trigger time measurement (walking strobe capture) tests from a pattern by including this opcode in the pattern file. Note that you cannot place instrument specific microcodes on the same vector as walking_strobe_start opcode, but you can program this opcode together with another PatGen opcode on the same vector. See Time Measurement Test Pattern Setup.

<		>	
	2015 Teradyne, Inc. All rights reserved.		

plmicrocode.4.12

<

>

Microcode (Opcodes and Control Bits) : Control Bits

Control Bits

Control bits act to modify the execution of a vector. They do not take arguments or depend on conditions.

You can place more than one control bit per vector. Control bits can appear on the same vector as opcodes.

You can program mask, care, and stv in every vector, even in vectors where opcodes are not permitted.

Failure-related Control Bits

Control Bit	Definition
mask	Sets the mask or ignore (IGN) bit of a vector to mask failures on the vector. When TheHdw.Digital.Patgen.MaskEnable = True, a vector marked as mask is not stored, even if it generates a fail. This allows you to mask or "ignore" parts of your pattern. This is commonly used with initialization code that can cause failing output on a device that is not yet in a working state. When using mask in your pattern, IG-XL will not perform the following actions: - Set the per-pin cumulative reject (CR) flag - Increment the per-pin fail counter - Halt the pattern if it is in halt on fail mode

- Store the cycle when the capture memories (HRAM, VM, deep scan capture memory, MTO CMEM) are in store fails mode
- Start HRAM storing in trigger on fail mode

You must place the mask keyword on all minor vectors of a major vector when executing patterns that require conditional branching, such as match loops, and using any timing mode other than single mode. Otherwise, IG-XL will not enable the PatGen fail flag when a fail occurs, which is essential for conditional branching.

care

Sets the CARE bit of a vector to enable failures on the vector. When TheHdw.Digital.Patgen.CareEnable = True, only vectors marked as care are stored when determining fails. This allows you to consider a selected portion of your pattern when testing for fails. See the mask control bit description above for actions related to fails and placement of care on all minor vectors of a major vector.

stv

Selects this vector for HRAM collection for certain HRAM modes. (Store This Vector.)

refresh_en

Sets the Interrupt Enable (INT_EN) bit of a major vector.

You can program refresh_en only in vectors where opcodes are allowed. So, when in dual mode, you can program refresh_en only in the second vector of each vector pair. When in quad mode, you can program it only in the fourth vector of each group of four vectors. When in By16 or By16IIO mode, you can program it only in the 16th vector of a 16-vector group.

You cannot use this control bit on same vector as the following opcodes:

- call SRM_IDENTIFIER
- if (branch_expr) call/jump SRM_IDENTIFIER
- return
- if (branch_expr) return

- [push_subr](#)
- [pop_subr](#)
- [clr_subr](#)
- [end_loop](#) SRM_IDENTIFIER

You cannot use this control bit on same VM vector as the following opcodes:

- [repeat](#) INTEGER
- [pipe_minus](#) INTEGER
- [repeat](#) COUNTER
- [wflag](#)
- [match](#) SRM_IDENTIFIER

You cannot use this control bit on same vector as these opcodes if the refresh subroutine itself uses either of these opcodes:

- [set_msb](#) INTEGER
- [set_msb_infinite](#)



plmicrocode.4.13

<

>

Microcode (Opcodes and Control Bits) : Conditional Statements

Conditional Statements

The pattern language permits conditional execution of opcodes. A set of condition flags can be enabled and then tested using the `if` opcode.

System Flags

The pattern language includes several flags that you can use for conditional pattern execution. During a pattern burst, the pattern generator monitors the flags. If one of the flags occurs, the corresponding condition latch is set.

System flags are:

fail	Set by the pattern generator when a vector fails during a pattern burst. The fail flag is cleared whenever it is read. Therefore, if a branch expression checks the fail flag, the flag is cleared after the <code>if</code> statement that checks the branch expression. There is no separate opcode to clear the fail flag.
pass	Not a separate flag, but the inverse of fail. Offered as a notational convenience. This flag cannot be cleared.
inst_cond	Indicates that an instrument has sent a flag to the Pattern Generator.

CPU Flags

In addition, the following CPU flags can be set or read from a test program or by opcodes in a pattern file:

`cpuA_cond`, `cpuB_cond`, `cpuC_cond`, `cpuD_cond`

On setting or reading CPU flags from a test program, see the Visual Basic for Test object PatGen. To set CPU flags from a pattern file, use the `set_cpu_cond` opcode. See [CPU Flag Condition Opcodes](#).

You must explicitly clear CPU flags. The `clr_cond_flags(flag-list)` opcode clears the specified flags unconditionally. Unlike system flags, CPU flags are not automatically cleared when a pattern start is executed.

The `inv_cond_flags(flag-list)` opcode inverts the specified flags.

branch_expr Opcode

Before using the `if` statement to test for one or more flags, you first use the `branch_expr` statement to set up the flags. Then you use the `if(branch_expr)` statement, where the `branch_expr` keyword specifies the condition that was set up by the most recently executed `branch_expr` statement.

The `branch_expr` syntax is:

`branch_expr=( flag-name )`

or

`branch_expr=( flag-name and flag-name ... )`

or

`branch_expr=( none )`

Note these points:

- You can negate any flag by using a ! (exclamation point) character before it.
- If there is more than one flag, you must specify `and` between each flag.
- Use `none` to clear all enabled conditions without setting new ones.
- If a second `branch_expr` occurs, the conditions set up by the first `branch_expr` are replaced by those specified in the second `branch_expr`. Only the latest `branch_expr` is in effect.

if Opcode

The `if(branch_expr)` statement tests conditions that were enabled by the most recently executed `branch_expr` statement. If the conditions are true, the command is executed.

If an `if(branch_expr)` is executed before its `branch_expr` is executed, `if(branch_expr)` uses the default condition pass.

You cannot use the modifier `refresh_en` on the same vector as an `if(branch_expr)` call/jump/return opcode.

The following opcodes can be conditionalized in either VM or SRM:

- `call`

- `jump`

The following opcodes can be conditionalized only in SRM:

•	<code>return</code>
---	---------------------

•	<code>jump_vm</code>
---	----------------------

<	>
-----------------	-----------------

plmicrocode.4.14

	
---	---

Microcode (Opcodes and Control Bits) : Instrument Microcode

Instrument Microcode

Many instruments have their own set of microcodes to perform actions using the instrument. Pattern control of instruments permits precise timing of instrument actions.

Instruments Statement

A pattern file must use the instruments statement to define the instrument before it can use microcode specific to the instrument. See [Instruments Statement](#).

General Instrument Syntax

Per-pin instruments use the same general format for specifying opcodes. The simplest form uses a statement in parentheses to assign an opcode to a pin:

```
(pin = opcode) > ...
```

For example, if pin A1 is attached to instr1, which has the opcode start, the syntax is:

```
(A1 = start) > ...
```

On a single vector, each instrument or instrument set that is programmed on the vector has its own set of parentheses:

```
(pin = opcode) (pin = opcode) > ...
```

For example, if instr2, connected to pin B1, is programmed on the same vector as A1, the syntax is:

```
(A1 = start) (B2 = stop) > ...
```

To use a list of pins that are connected to the same instrument type, enclose the list of comma-separated pins in another set of parentheses:

```
((pin, pin, pin) = opcode)
```

For example, if A1 and A2 are both connected to instr1 instruments, the syntax is:

```
( (A1, A2) = start) > ...
```

If you are using multiple opcodes with a single pin or pin list, use a comma-separated list of opcodes:

```
(pin = opcode, opcode, opcode)
```

For example:

(A1 = start, gate_on) > ...

If multiple resources are behind a pin, you must include the instrument type as well as the pin, to specify which instrument the microcode applies to:

(pin:instrument = opcode)

For example, if A1 is connected to instr1 as the primary instrument and to instr2 through DIB access, you must specify which instrument is intended:

(A1:instr2 = start) > ...

Note that you can always append :instrument to a pin name, even if there are not multiple resources behind the pin. It is required when there are multiple resources behind a pin.

Nonpin instruments use a different basic syntax:

(instrument opcode-list) > ...

For details, see the programming sections for the instrument.

Specific Instrument Syntax

For information on microcodes for a specific instrument, click [Related Topics](#).

< >		
	2015 Teradyne, Inc. All rights reserved.	

plmicrocode.4.15

<	>
---	---

Microcode (Opcodes and Control Bits) : Microcode Groups

Microcode Groups

A vector in a pattern consists of pattern microcode and channel data for each pin in the pin list for that vector. Since a vector pinlist can include multiple instruments, this means that a vector can include a number of microcodes for both digital and analog instruments. Each unique set of microcodes is considered a **microcode group**.
IG-XL coordinates the activity of vectors and microcodes using ICS (instrument control set) tables. The control mechanism in V7.40 and later is called "dynamic ICS," and it allows:

- Up to 249 unique microcode groups per pattern start in each time domain
- Up to two Resyncs per time domain

Dynamic ICS, therefore, allows you to create thousands of unique microcode groups in many patterns inside multiple time domains. This provides excellent capability for running large mixed-signal pattern tests and for running tests concurrently.

How ICS Works

ICS is an IG-XL software process that runs in the background, and you do not need to enable or control it. This section describes how ICS works.
In the standard ICS process, messages are used by IG-XL to provide synchronized control of instruments. For example, in a pattern with microcodes at different vectors, ICS messages are sent from the digital instrument's pattern generator to an analog instrument. Thus, the analog instrument will perform the designated operation at a specific vector.
When compiling a pattern with instrument microcodes, the Pattern Compiler assigns an ICS value (a code number) to each unique microcode combination for vectors in the pattern module. For example, a pattern module might have the following content:

Example Pattern Module

Vector	Instrument A	Instrument B	Instrument C
0	PSet Set_1	PSet Set_1	PSet Set_1
1			
2			
3			
4	Strobe	Strobe	Strobe
5			
6			
7	PSet Set_2		
8			
9			
10			
11	PSet Set_2		
12			
13			
14			
15	Strobe	Strobe	
16			
17			
18			
19			PSet Set_3
20			
21			
22			
23	Strobe		Strobe

After a pattern is compiled, an ICS table is generated locally in the pattern module. the following is an example of an ICS table for the example above:

Example ICS Table

ICS Value	Instrument A	Instrument B	Instrument C	Comment
0				Reserved for null
1	uCode_1	uCode_1	uCode_1	From Vector 0
2	uCode_2	uCode_2	uCode_2	From Vector 4
3	uCode_3			From Vectors 7 and 11
4	uCode_2	uCode_2		From Vector 15
5			uCode_4	From Vector 19
6	uCode_2		uCode_2	From Vector 23

You can see that most vectors in this example pattern are unique. The one exception is vectors 7 and 11, which have the same microcode combination. This microcode combination is counted as one unique combination, even though it is used twice.

When all patterns in a test program are loaded, all local ICS tables are consolidated into a single table of ICS global values. The same microcode combinations share the same ICS value that they had in the ICS table for each pattern module. The proper mapping is generated and updated to each of the instruments.

When patterns in the test program are executed, ICS values are sent to each instrument using ICS messages. The instruments get each ICS value and perform the mapped microcode at the specified vector time. If the test program is using time domains and running tests concurrently, then the mapped microcodes will be run at the time dictated by each time domain.

This allows you to include up to 249 unique microcode groups, including two Resyncs, in a test program.

Dynamic ICS adds the following features:

- The ability for IG-XL to detect when a test program uses more than 249 unique microcode groups in a test program
- The ability to generate a table of ICS global values per burst per time domain
- The ability to detect instruments that support extended microcode of more than 256 microcode groups

This means that IG-XL can automatically determine the number of unique microcode groups each time patterns are executed for every time domain. If the total number for the test program exceeds

249, IG-XL can reload the ICS tables to expand the microcode group limit. This allows each test program to include up to 249 microcode groups per pattern start per time domain and up to two Resyncs per time domain. As with standard ICS, you do not need to do anything to your test program to use this feature.

Instrument Limitations

Although IG-XL and dynamic ICS support thousands of unique microcode groups in a test program, UltraFLEX instruments differ in their ability to use this capability. Each instrument has a limitation on how many microcodes it can support for each pattern burst. For example, each DC30 channel can support 60 microcodes per burst, and each BBAC channel can support 64. For more information, see the hardware specifications for each instrument.

plnotes

	
---	---

Sample Pattern

Sample Pattern

The following is a sample test pattern file using dual mode for the HSD1000 digital instrument:

```
import tset ts0, ts1, ts2;
import subr sub0;
pinmap_workbook = "BVH90210.xls";
version = "Version 1.0";
digital_inst = hsdm;
opcode_mode = dual;
scan_type = x6;
cdl_cache = no;    // Turn off CDL Caching

pin_setup = {
    pin1    window;
}
scan_pins = {
    sp0, sp1, sp2, sp3
}
// VM VECTORS
vm_vector MODULE0 ( $tset pin0 pin1 abus:X (sp0,sp2)(sp1,sp3)){
start_label mypat0:
mask          >  ts0    1    H    .dff    .0    .L  ;
call sub0 mask >  ts0    1    H    .dff    .1    .H  ;
mask          >  ts0    1    H    .dff    .0    .L  ;
mask          >  ts0    1    H    .dff    .1    .H  ;
// NOTE: There are more vectors here to meet minimum size of 128 vectors
//      in a VM module.
```

...

```
mask          > ts0  1  H  .dff  .0  .L  ;
```

```
branch_expr = (fail)
```

```
    > ts0  1  H  .dff  .1  .H  ;
```

```
    > ts1  0  L  .sxxxxxxx .0  .L  ;
```

```
repeat 1024 stv > ts1  0  L  .sxxxxxxx .1  .H  ;
```

```
    > ts0  1  H  .raa  .0  .L  ;
```

```
if (branch_expr) call mysub
```

```
    > ts0  1  H  .raa  .1  .H  ;
```

```
scan_setup    > ts1  0  X  .sxxxxxxx .0  .L  ;
```

```
scan  36      (si sp0:X > ABCDEF123)
```

```
    (so sp1:X > FF0EEFFA)
```

```
    (si sp2  > [36-1])
```

```
    (so sp3  > [36-H]);
```

```
mask          > ts1  0  L  .sxxxxxxx .0  .L  ;
```

```
halt          > ts1  0  L  .sxxxxxxx .1  .H  ;
```

```
}
```

```
// SRM VECTORS:
```

```
srm_vector MODULE1 ( $tset pin0 pin1 abus:X (sp0,sp2)(sp1,sp3)){
```

```
global subr mysub:
```

```
    > ts0  1  H  .dee  .0  .L  ;
```

```
set_msb 0     > ts0  1  H  .dee  .1  .H  ;
```

```
    > ts0  1  H  .dee  .0  .L  ;
```

```
set c4 50000  > ts0  1  H  .dee  .1  .H  ;
```

```
    > ts0  1  H  .dee  .0  .L  ;
```

```
loop c4       > ts1  0  W  .dab  .1  .H  ;
```

```
check_lp:
```

```
    > ts0  1  H  .dee  .0  .L  ;
```

```
    > ts1  0  W  .dab  .1  .H  ;
```

```
    > ts0  1  H  .dee  .0  .L  ;
```

```
repeat 200    > ts1  0  W  .dab  .1  .H  ;
```

```
    > ts0  1  H  .dee  .0  .L  ;
```

```
end_loop check_lp > ts1  0  W  .dab  .1  .H  ;
```

```
    > ts0  1  H  .dee  .0  .L  ;
```

```
return        > ts0  1  H  .dee  .1  .H  ;
```

```
}
```

< >		
	2015 Teradyne, Inc. All rights reserved.	

ploverview.1.2



About Pattern Language : Summary of Pattern Language Statements

Summary of Pattern Language Statements

Types of Pattern File Contents

A pattern file has these major parts:

header statement	Contains information required to compile the pattern, such as the pin map where the pins are defined, the digital instruments, compiler statements, and labels and timing set names.
setup statement	Defines special digital pin and scan pin setups, plus any analog instruments that are used.
pattern module	Contains the pin data that comprises the bulk of the compiled pattern. A pattern file can contain several pattern modules.

Note that these parts must appear in the ASCII pattern file in the order shown here. See [Pattern File Order](#).

A pattern file can also contain comments and preprocessor statements.

The following sections define these parts of a pattern file.

Header Statements

The following statements belong in the header section of an ASCII pattern file:

Digital Instrument Statement	This statement specifies the name of the digital instrument. This information is required and can come from this statement in the file or from a compile-time option.
Pin Map Specification	<code>pinmap</code> statements specify the workbook and sheet where the pins and pin groups are defined.

	This information is required and can come from these statements in the file or from a compile-time option.
Compiler Control Statements	These are pattern compiler options, also available at compile time.
Import Tset or Label Statement	The import statement imports tset (time set) names and any external labels. These imports pertain to all pattern modules in the pattern file.

Setup Statements

The following statements belong in the setup section of an ASCII pattern file:

Pin_setup Statement	A pin_setup statement defines any special digital pin setups.
Instruments Statement	An instruments statement specifies any instruments that have extended microcode used in the pattern file.
Scan_pins Statement	A scan_pins statement specifies setup information for scan pins.

Setup statements apply to all the pattern modules in the pattern file.

Pattern Modules

A pattern file must contain one or more pattern modules. A pattern module is set of vectors that, when the pattern file is compiled, can be specified independently of the pattern file. See [Pattern Modules](#).

A pattern module contains a pin list (see [Pin List](#)) and individual vectors. All pattern modules in a pattern file must have identical pin lists.

Each vector can include:

•	One or more labels (see Vector Labels)
•	An opcode and/or control bits (see Opcodes and Control Bits Overview)
•	A time set (tset) field (see Tset (Time Set) Data)
•	Vector data (see Symbolic and Numeric Pin Data)

Preprocessing Statements

A pattern file can contain C++ preprocessor statements. These can occur anywhere in the ASCII pattern file. See [Preprocessor Statements](#).

Comments

Any pattern file statement can include optional C++-style comments. See [Comments in Pattern Files](#).

<		>	
	2015 Teradyne, Inc. All rights reserved.		

ploverview.1.3

	
---	---

About Pattern Language : Pattern File Format Requirements

Pattern File Format Requirements

Each ASCII pattern file has a required format.

Format and Delimiters

All statements are in free format.

Elements are delimited by white space, including tabs and line breaks.

Line breaks have no special significance in the syntax. For readability, any statement can span multiple lines.

Pattern File Order

Statements in a pattern file must appear in a certain order. Specifically, groups of statements must appear in a certain order. Individual statements within a single group can appear in any order within that group.

The order in which statement groups must appear is as follows:

- | | |
|----|--|
| 1. | Header statements: digital instrument, pin map, compiler control, import |
| 2. | Setup statements: pin setup, instruments, scan pin statements |
| 3. | Pattern modules |

Comments and preprocessor statements can appear anywhere in a pattern file.

Case

PatGen opcodes and control bits are case-sensitive. See [List of Opcodes and Control Bits](#).

In all other cases, the pattern compiler is case-insensitive. You can use uppercase and lowercase interchangeably.

<		>	
	2015 Teradyne, Inc. All rights reserved.		

ploverview.1.4

<	>
-----------------	-----------------

About Pattern Language : Pattern File Filename and Pathname

Pattern File Filename and Pathname

A pattern file’s filename and pathname cannot contain the characters “+” (plus sign) or “,” (comma). If either of these characters appears anywhere in a pattern file’s filename or in the directory names that make up the pathname to the pattern file, IG-XL will not be able to find the file.

The reason is that the plus sign and comma have a special meaning with respect to patterns. The plus sign is used to indicate that pattern files are to be executed concurrently. The comma is used to separate the items in a list of pattern files. IG-XL therefore interprets everything up to the plus sign or comma as a complete pattern file pathname and filename, and everything after the plus sign or comma as another pattern file.

In addition, the pattern file name must be different from the names of all pattern sets and pattern modules used in the job. Because the pattern file name includes the .pat extension, it should never duplicate the name of a pattern set or module.

If a pattern contains only a single module, the module can take a default module name based on the pattern filename. In this case, the pattern filename must conform to the requirements for a module name, which are more restrictive than those for a pattern filename. See Module Name.

<	>
-----------------	-----------------

ploverview.1.5

<

>

About Pattern Language : Syntax Notation

Syntax Notation

Pattern language descriptions use this syntax notation:

courier	Enter this keyword or symbol exactly as shown. Note that all keywords are lowercase.
italics	This is a placeholder, for example, filename or integer. Replace it with the appropriate value.
	"Or"; choose one of the items separated by vertical lines.
[aaa]	The item in square brackets is optional. The square brackets are not part of the item.
...	The preceding item is repeated any number of times.

<

>

ploverview.1.6

<	>
---	---

About Pattern Language : Comments in Pattern Files

Comments in Pattern Files

Comment Formats

The pattern compiler accepts C++-style comments:

- The form that includes everything between the delimiters:

```
/* comment-text */
```

- The end-of-line form that is terminated by a carriage return:

```
// comment-text
```

Saving Comments in a Compiled File

You can specify that the comments in the source .atp file should be retained in the output .pat file. If you save the comments, PatternTool can display them and the pattern reverse compiler writes them to the output .atp file. You can also specify that comments should be stripped from the compiled .pat file. This lets you minimize file size.

You specify the option you want using one or more of these interfaces:

- the `save_comments = no | yes` compiler control statement in the .atp source file. See [save_comments](#) in the section on compiler control statements.

If you omit the statement, the default for the .atp file is not to save comments. This default can be overridden by the other interfaces.

- the Save Comments check box on the Pattern Compiler window. See [Save Comments](#) in the section on the Pattern Compiler.

The Save Comments box overrides whatever is in the .atp source file. If the box is checked, comments are saved. If it is not checked, they are discarded.

- the -comments switch on the Pattern Compiler command line interface. See [Pattern Compiler Command-Line Interface](#).

If you include the switch, comments are saved, regardless of the setting in the .atp source file. If you omit the switch, the action depends on the setting in the .atp file.

- the save_comments property on the Pattern Explorer Properties panel in PatternTool. See [Viewing and Changing Pattern Properties](#) in the PatternTool section.

If you add comments using PatternTool, you must make sure that the property is set to yes for the comments to be saved when you save the file.

Notes on Comments

If comments are saved, the pattern compiler retains information about the type of a comment, whether it's a block comment (`/* */`) or an end-of-line comment (`//`). The distinction between block comments and end-of-line comments is preserved during reverse-compilation, so they appear as the correct type of comment in the output .atp file.

PatternTool displays end-of-line comments in a separate Comment column.

There are several points to be noted about block comments, as described in the following paragraphs.

- A block comment is associated with or "belongs to" the vector that follows it.

```
> ts0  L  L  L  L  L  LLLLLL ... ;
/* This is before the second vector */
```

```
> ts0  X  0  0  X  X  000000 ... ;
```

The block comment is associated with the second vector shown here.

- A comment in the middle of a vector is moved to a separate row before the vector. For example:

```
> ts0  L  L  L  L  L  LLLLLL ... ;
> ts0  X  0  /* Comment in middle */ 0  X  X  000000 ... ;
```

In PatternTool or when reversed compiled, the comment is moved to the row before the vector that it is associated with:

```
> ts0  L  L  L  L  L  LLLLLL ... ;
/* Comment in middle */
```

```
> ts0  X  0  0  X  X  000000 ... ;
```

- If a vector has multiple block comments, they are grouped together as a single comment. For example:

```
> ts0  L  L  L  L  L  LLLLLL ... ;
/* This is before the second vector */
/* This is also before the second vector */
> ts0  X  0  0  X  X  000000 ... ;
```

In PatternTool or when reversed compiled, the two comments are combined into a single comment, with a single opening and a single closing symbol:

```
> ts0  L  L  L  L  L  LLLLLL ... ;
/* This is before the second vector
   This is also before the second vector */
> ts0  X  0  0  X  X  000000 ... ;
```

- All comments before the vector definitions in a module (that is before the { that starts vectors), including any before the pattern module, are combined into one comment, known as the Start Comment. Each module can have a Start Comment. PatternTool does not display Start Comments.

For example:

```
opcode_mode = Single;
/* This is before any of the compiler statements */
import tset ts0, ts1, ts2, ts3, ts4;
vm_vector
/* This is before the pin list */
mod1_FirstPattern ( $tset, P0, P1, P2, P3, ... )
{ ...
```

When reversed compiled, the comments are combined at the beginning of the file:

```
/* This is before any of the compiler statements
   This is before the pin list */
opcode_mode = Single;
import tset ts0, ts1, ts2, ts3, ts4;
vm_vector
mod1_FirstPattern ( $tset, P0, P1, P2, P3, ... )
{ ...
```

Note that a block comment after the final vector in a module becomes part of the Start Comment for the next module. This is because a block comment is associated with the vector that follows it.

Therefore, any block comment after the final vector in a module (except the last module - see the next bullet on Final Comment) is associated with the next module. For example:

```
> ts0  L  L  L  L  L  LLLLLL ... ;
/* This is before the final vector of the first module */
> ts0  X  0  0  X  X  000000 ... ;
/* This is after the final vector of the first module */
}
vm_vector
/* This is after the vm_vector statement of the second module */
mod2_FirstPattern ( $tset, P0, P1, ...)
```

```
{
/* This is before the first vector of the second module */
```

```
> ts0  0  0  0  0  0  000000 ... ;
```

When reversed compiled, the comment after the final vector of the first module and the comment before the { in the second module are combined into a Start Comment for the second module:

```
> ts0  L  L  L  L  L  LLLLLL ... ;
/* This is before the final vector of the first module
> ts0  X  0  0  X  X  000000 ... ;
}
/* This is after the final vector of the first module
This is after the vm_vector statement of the second module */
vm_vector
mod2_FirstPattern ( $tset, P0, P1, ...)
```

- All comments after the last vector of the last pattern module in the .atp file are combined into one comment, known as the Final Comment. PatternTool does not display the Final Comment.

<

>

ploverview.1.7

<	>
---	---

About Pattern Language : Preprocessor Statements

Preprocessor Statements

The pattern language supports all C++ preprocessor constructs, such as `#include`, `#ifdef`, and `#define`.

The Pattern Compiler does not automatically run the C++ preprocessor; you must invoke it using a compile-time option (Run Preprocessor option or `-cpp` switch).

The Pattern Compiler also provides a means to pass macro definitions to the preprocessor.

If you do not use any preprocessor constructs other than comment, do not run the C++ preprocessor. This speeds up pattern compilation.

For more information, see [Interface Overview](#) and [Pattern Compiler Command-Line Interface](#) in the Pattern Compiler section.

<	>
---	---

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

ploverview.1.8



About Pattern Language : Reserved Words in Pattern Files

Reserved Words in Pattern Files

User-defined symbols used in pattern files (such as pin names, pin group names, labels, or tset names) cannot use any of these words:

compressed

datacodes

defaults

digital_inst

download

global

import

import_all_undefines

instruments

keepalive

label

multiinst

no

opcode_mode

output_filename

pin_setup

pinmap_workbook

pinmap_sheet

scan

scan_pins

scan_setup

scan_type

si

so
srm_vector
start_label
stop
subr
svm_subr
to
tset
\$tset
vector
version
vm_vector
yes

In addition, a user-defined symbol cannot use any word that is the name of an opcode or a control bit. See [List of Opcodes and Control Bits](#).

< >		
	2015 Teradyne, Inc. All rights reserved.	

plvectors.3.01

<

>

Modules, Vectors, and Vector Data

Modules, Vectors, and Vector Data

Vectors make up the body of a pattern module. The following topics are available:

- [Pattern Modules](#)
- [Pattern Module Requirements for VM and SRM](#)
- [Pin List](#)
- [Vector Labels](#)
- [Relative Addressing](#)
- [Subroutines](#)
- [Tset \(Time Set\) Data](#)
- [Symbolic and Numeric Pin Data](#)
- [Initialization Pattern](#)
- [Stop Subroutines](#)

- [Keepalive Subroutines](#)

For information on the microcode portion of a pattern module, see [Microcode \(Opcodes and Control Bits\)](#).

<

>

2015 Teradyne, Inc. All rights reserved.

plvectors.3.02



Modules, Vectors, and Vector Data : Pattern Modules

Pattern Modules

A pattern file can contain one or more pattern modules. A pattern module is a named, demarcated set of vectors in a pattern file. For the HSDM (HSD1000 and UltraPin800), HSDMQ (UltraPin1600), and HSD4G (UltraPin4000 and HPM) digital instruments, a pattern module is explicitly assigned either to Vector Memory (VM) or to Subroutine Memory (SRM). Any opcodes that are SRM-only must go into an SRM pattern module. When including scan vectors, you must use VM pattern modules only. IG-XL returns an error if you include scan vectors in an SRM module.

Once a pattern file is compiled, each pattern module can be specified independently from other modules in the pattern file. Pattern modules from several pattern files can be combined into a single pattern set that can then be executed as a single burst.

For more information, see Pattern Module Requirements for VM and SRM.

Syntax

For the HSDM, HSDMQ, and HSD4G digital instruments, the syntax for a pattern module is as follows.

For a pattern module that is to go into Vector Memory (VM):

```
vm_vector [ module-name ] ( pin-list )
    { vectors }
```

For a pattern module that is to go into Subroutine Memory (SRM):

```
srm_vector [ module-name ] ( pin-list )
    { vectors }
```

For details, see the following sections.

Module Name

If a pattern file contains more than one pattern module, the module name is required for every module.

A module name must be a character string consisting only of alphanumeric characters and the underscore. It must start with an alphabetic character, not with a number. Embedded blanks are not

permitted.

All pattern module names used in a single test program must be unique, even if they appear in separate pattern files. The pattern loader enforces this uniqueness requirement. In addition, a pattern module cannot have the same name as any pattern set that is used in the job. The names of pattern modules and pattern sets must also be different from the names of all pattern files in the job; because the .pat extension is part of the pattern file name, duplication of a pattern set or module name with a pattern file name is unlikely.

If a pattern file contains only one pattern module, the name is optional. If it is omitted, the default module name is the base name of the output file (the output filename without the extension), plus an underscore (_) and the suffix "module". For example, the default module name for the pattern file "MyPat.pat" would be "MyPat_module." Note that the resulting default module name must conform to the rules stated above for all module names. This means that the pattern file name, which is used to create the default module name, must contain only alphanumeric characters and the underscore, and must begin with an alphabetic character.

Pin List and Tsets

The pinlist assigns pins to the vector data that follows the pinlist. A tset (time set) value can be part of the pin list; it can also be part of the vector data.

For more information, see [Pin List and Tset \(Time Set\) Data](#).

Vectors

A vector consists of the pattern microcode and channel data associated with each pin or pin group in the vector pinlist.

All vector data for a pinlist is enclosed in curly braces. Each vector is terminated with a semicolon. A vector can span multiple lines; end of line is not significant.

A single vector in a pattern module has the following fields (from left to right):

- | | |
|---|---|
| • | One or more labels (optional). See Vector Labels . |
| • | An opcode and/or control bits (optional). See Opcodes and Control Bits Overview . |
| • | The greater than (>) delimiter (required) |
| • | A time set (tset) field (optional). See Tset (Time Set) Data . |
| • | The vector data fields (required). See Symbolic and Numeric Pin Data . |
| • | The terminating semicolon (;) (required) |

A pattern module can contain one or more vectors with this format.

<		>	
	2015 Teradyne, Inc. All rights reserved.		

plvectors.3.03

<	>
-----------------	-----------------

Modules, Vectors, and Vector Data : Pattern Module Requirements for VM and SRM

Pattern Module Requirements for VM and SRM

For all digital instruments, a pattern module is explicitly assigned to Vector Memory (VM) or Subroutine Memory (SRM).

Pattern sets can contain VM modules only, SRM modules only, or a combination of VM and SRM modules. For more information on bursting pattern modules from the Pattern Sets sheet, see [Bursting VM and SRM Pattern Modules](#).

For information declaring a VM or SRM pattern module, see [Pattern Modules](#).

HSDM (HSD1000 and UltraPin800)

Note the following requirements for VM and SRM when using the HSDM digital instruments:

- | | |
|--|--|
| | <ul style="list-style-type: none">• An SRM pattern module cannot have more than 1023 vectors in single mode or 2046 vectors in dual mode. |
| | <ul style="list-style-type: none">• A VM pattern module cannot have more than 64M vectors in single mode or 128M vectors in dual mode. |
| | <ul style="list-style-type: none">• A VM module must contain at least 64 vectors in single mode and 128 vectors in dual mode. |
| | <ul style="list-style-type: none">• For all but the last VM module in a VM pattern set, IG-XL has a minimum cycle count per module. The module must generate enough cycles to let the system update the pointer to the next VM module in the pattern set. In single mode, the module must generate at least 512 cycles. In dual mode, it must generate 1024 cycles. The cycles can be generated from individual vectors or from repeats. This restriction does not apply to SRM-only pattern sets. |

- The last VM module in the pattern set must contain at least 64 vectors, but there is no minimum cycle count.

HSDMQ (UltraPin1600)

Note the following requirements for VM and SRM when using the HSDMQ digital instrument:

- An SRM pattern module cannot have more than 1023 vectors in single mode, 2046 vectors in dual mode, or 4092 vectors in quad mode.
- A VM pattern module cannot have more than 128M vectors in single mode, 256M minor vectors in dual mode, or 256M minor vectors in quad mode.
- A VM module must contain at least 64 vectors in single mode, 128 vectors in dual mode, and 256 vectors in quad mode.
- For all but the last VM module in a VM pattern set, IG-XL has a minimum cycle count per module. The module must generate enough cycles to let the system update the pointer to the next VM module in the pattern set. In single mode, the module must generate at least 512 cycles. In dual mode, it must generate 1024 cycles. In quad mode, it must generate 2048 cycles. The cycles can be generated from individual vectors or from repeats. This restriction does not apply to SRM-only pattern sets.
- The last VM module in the pattern set must contain at least 64 vectors, but there is no minimum cycle count.

HSD4G (UltraPin4000 and HPM)

Note the following requirements for VM and SRM when using the HSD4G digital instruments:

- An SRM pattern module cannot have more than 1023 vectors in By1 mode or 16368 vectors in By16 mode.
- A VM pattern module cannot have more than 32M vectors in By1 mode or 512M vectors in By16 mode.
- A VM module must contain at least 64 vectors in By1 mode and 128 vectors in By16 mode.

- For all but the last VM module in a VM pattern set, IG-XL has a minimum cycle count per module. The module must generate enough cycles to let the system update the pointer to the next VM module in the pattern set. In By1 mode, the module must generate at least 512 cycles. In By16 mode, it must generate 1024 cycles. The cycles can be generated from individual vectors or from repeats. This restriction does not apply to SRM-only pattern sets.
- The last VM module in the pattern set must contain at least 64 vectors, but there is no minimum cycle count.

plvectors.3.04

	
---	---

Modules, Vectors, and Vector Data : Pin List

Pin List

A pin list assigns pins to the vector data that follows the pin list.

Syntax

The pin-list syntax is:

```
([$tset,] pin-item [, pin-item ] ... )
```

The syntax items are explained in the sections below.

Multiple Pattern Modules

If a pattern file contains more than one pattern module, all modules must have identical pin lists.

That is, even though vectors may be in different pattern modules, they contain data for the same list of pins.

Notes on \$tset

\$tset is a keyword that designates the column in which the tset number or name will appear. The \$tset keyword can appear anywhere in the pin list, not only as the first item. If omitted from the pin list, a tset is specified for each vector in the statement. See [Tset \(Time Set\) Data](#).

Notes on pin-item

A pin-item has this syntax:

```
pin-or-group[.modifier][:radix]
```

pin-or-group can be:

- | | |
|---|---|
| • | A pin name or pin group name (defined in the Pin Map sheet) |
| • | Any combination of pins and pin groups, enclosed in parentheses and separated by commas, in the form: |

(item, item, ..., item)

You can group pins within a pinlist by enclosing them in parentheses. This packs together a number of pins, so that the pin data can be specified in a single column for all pins in the group. Separate individual items in such a group by commas.

- If a pin item is a pin group, every pin in the pin group must have the same pin setup.

modifier specifies components of a differential pin pair or group. The modifier is appended to the pin or group name with a period. Possible modifiers are:

"A"	Specifies the true side of a differential pair/group: for example, DBUS.A, where DBUS is a differential group.
"B"	Specifies complement side of a differential pair/group: for example, DBUS.B.
"C"	Specifies common mode voltage for a differential pair/group: for example, DBUS.C.
"D"	Specifies differential voltage for a differential pair/group: for example, DBUS.D.

radix indicates the format of the data for a pin or group. If a radix is not specified, the default for the pin or pin group is symbolic. The radix is appended to the pin item with a colon. Possible values are:

X or H	Hexadecimal
Q or O	Octal
D	Decimal
B	Binary
S	Symbolic (default)

The radix determines how data for the pin or group is to be interpreted. You can override the radix and use symbolic data with pins that have a numeric radix, and numeric data with pins that have a symbolic data.

For more information, see Symbolic and Numeric Pin Data.

Commas and White Space

Commas are optional between items in a pin list. If commas are used, do not include a comma between the last item and the closing right parenthesis.

White space is permitted anywhere in a vector pinlist, including around the colon used to separate a pingroup from a radix.

This is a sample pin list:

```
vm_vector vmmodule1 ($tset, (p4, p68), p13, abus:X, (p22, p23, p24):O)
{ vector-data }
```

<		>	
	2015 Teradyne, Inc. All rights reserved.		

plvectors.3.05

	
---	---

Modules, Vectors, and Vector Data : Vector Labels

Vector Labels

Label Format and Placement

The label format is as follows:

- A label name must consist only of letters, digits, and the underscore character (_).
- The label name must start with a letter.
- Labels are not case-sensitive.
- The label must be followed immediately by a colon, with no intervening white space.
- A given vector can have multiple labels.
- In the ASCII source file, because line breaks have no special significance, labels can be on the same line as the vector data, or for readability they can be put on a separate line before the vector data.

Labels can appear only on the first vector of a vector group:

- In single or By1 mode, labels can occur on any vector.
- In dual, quad, or By16 mode, labels can occur only on the first vector of a vector group.

Note the following restrictions:

- Subroutine labels (labels of type `subr`, `global subr`, `stop subr`, and `keepalive subr`) are allowed only in SRM modules.
- At most, one `stop subr` and one `keepalive subr` is allowed in an SRM pattern module.

The scope of a local label is the pattern module in which it is defined. That is, if an opcode references a label, the label must be defined in the current pattern module as a local label, or defined as a global label in another pattern module in the same pattern file, or imported with an `import` statement in the pattern files. (See `Import Tset` or `Label Statement`.)

Kinds of Labels

There are two kinds of labels: those that are used to begin subroutines (allowed only in SRM modules), and more general labels. These are described in the next two sections.

General Labels

The following are the labels other than those used to begin subroutines:

<code>start_label label-name</code> :	A label that is to be used only for pattern start or stop. Start labels are automatically exported so the test program can access them. The first vector of a pattern does not require a label. Pattern start defaults to the first vector of a pattern file if no label is specified. In VM, if a start label is to be used as the starting point for pattern execution, it must be on the first vector of the module; otherwise, it can be used only as the stopping point for execution. In SRM, a start label can be on any valid vector in the module.
<code>label-name</code> :	A label that is to be referenced only within the same pattern module. A local label has no keyword prefix, just the label name followed by a colon. A local label cannot be referenced by the test program or other pattern modules. A local label cannot have the same name as the module in which it appears. The pattern loader enforces this uniqueness requirement.
<code>global label-name</code> :	A label that is to be referenced by other pattern modules. A global label can be referenced by the test program as well as other pattern modules in the pattern file or in other pattern files. A global label cannot have the same name as any module that is loaded at the same time as the module containing the global label. The pattern loader enforces this uniqueness requirement.

Note that stop labels do not have a separate label type. Instead, you can use labels of other types as stop labels. The type of label depends on whether the stop label is used in SRM or VM:

- In an SRM module, a stop label can be declared as a global label or as a start label.
- In a VM module, a stop label can be declared only as a start label.

Labels That Begin Subroutines

The other labels are used to begin subroutines. Note that subroutine labels are allowed only in SRM pattern modules. Subroutine labels are:

subr	A label that is to be called as a subroutine from within this pattern
subroutine-name :	module.
global subr	A subroutine that needs to be accessible by other pattern modules or by
subroutine-name :	the test program.
stop subr	A stop subroutine. Stop subroutines are assumed to be global. An SRM
subroutine-name :	pattern module can contain at most one stop subroutine.
keepalive subr	A keepalive subroutine. Keepalive subroutines are assumed to be global.
subroutine-name :	An SRM pattern module can contain at most one keepalive subroutine.
	If one subroutine in a pattern module is declared as a keepalive
	subroutine, the whole pattern module is treated as keepalive.

For more information, see Subroutines.

Interpattern Label References

A pattern can reference global labels in another pattern file, either calling a global subr label or jumping to a global label. The reference to an external label does not include the name of the other pattern file. Instead, interpattern label references are resolved by the pattern linker, which looks for label references among all loaded patterns.

Relative Addressing

Any opcode that takes a label as an operand and is in an SRM pattern module can use a relative address instead of a label. See Relative Addressing.

<

>

plvectors.3.06

<

>

Modules, Vectors, and Vector Data : Relative Addressing

Relative Addressing

Opcodes that take an SRM label as an operand, such as call, jump, or match, can use a relative address instead of a label. Note these restrictions:

- Relative addressing is permitted only in SRM pattern modules.
- The address must resolve to a vector within the current SRM pattern module.
- For dual mode patterns, the address must resolve to the first vector in a vector pair, since you cannot execute the second vector in a vector group without executing the first vector as well.
- For quad mode patterns (UltraPin1600 only), the address must resolve to the first vector in a four-vector group, since you cannot execute all vectors in a vector group without executing the first vector as well.
- For By16 mode patterns, the address must resolve to the first vector in a 16-vector group, since you cannot execute all vectors in a vector group without executing the first vector as well.

Format

Possible relative addresses are:

.	(Period) Refers to the current vector.
.+N	Moves execution down N vectors from the current vector. N is decimal.

.-N

Moves execution up N vectors from the current vector. N is decimal.

When using .+N or .-N, no white space is allowed after the period or before the N.

<

>

2015 Teradyne, Inc. All rights reserved.

plvectors.3.07



Modules, Vectors, and Vector Data : Subroutines

Subroutines

A subroutine is a set of vectors introduced by a subroutine label. See [Vector Labels](#).

For information on calling and returning from a subroutine, see [Subroutine Opcodes](#).

A special kind of SRM subroutine is the keepalive subroutine. See [Keepalive Subroutines](#).

The stop subroutine is a type of vector module that executes at the end of a pattern burst. See [Stop Subroutines](#).

Rules for Subroutines

Subroutines in a pattern file must follow these rules:

- [Subroutines in srm_vector modules only](#)

Subroutines can appear only in Subroutine Memory, that is, in `srm_vector` pattern module.

- [Declaring subroutine labels](#)

Subroutine labels must be declared explicitly as subroutines, using one of these label prefixes: `subr`, `global subr`, `stop subr`, or `keepalive subr`. See [Vector Labels](#).

- [Subroutines in another pattern file](#)

If a called subroutine is in another pattern file, the called subroutine label must first be imported using the `import` statement. A subroutine in the same pattern file as the calling vector does not need to be imported. See [Import Tset or Label Statement](#).

- [Pattern Subroutine sheet in DataTool](#)

To facilitate calling subroutines, DataTool supports a Pattern Subroutine sheet. This sheet lists patterns containing subroutines that can be called from other patterns. Listing the patterns on the sheet guarantees that the patterns are loaded correctly at pattern start time. See [Pattern Subroutine Sheet](#) in the DataTool section.

- [Instruments statements in subroutines](#)

If a pattern file defines an instrument in an instruments statement, and if it calls subroutines in another pattern file, the subroutine pattern file must have the same instrument setup, even if it does not use any instrument microcode. See [Instruments Statement](#).

- [Number of subroutines called](#)

There is a limit of 1023 user subroutines that can be called from VM. If you attempt to validate a workbook whose patterns call more than 1023 subroutines, a validation error is returned.

- [Subroutine stack](#)

For the HSDM digital instruments (HSD1000 and UltraPin800), the maximum depth of the subroutine stack is 16.
For the HSDMQ digital instrument (UltraPin1600), the maximum depth of the subroutine stack is 16.
For the HSD4G digital instruments (UltraPin4000 and HPM), the maximum depth of the subroutine stack is 16.

- [Use of the HALT opcode](#)

You must include a HALT opcode in the subroutine if the last executed vector in a pattern is in SRM. This prevents execution from continuing past the end of the module.

<

>

2015 Teradyne, Inc. All rights reserved.

plvectors.3.08

<	>
-----------------	-----------------

Modules, Vectors, and Vector Data : Tset (Time Set) Data

Tset (Time Set) Data

Tset names, which are defined in the program workbook, must be imported using the `import tset` statement before they are used in the pattern module. Tset names are resolved when the pattern is loaded. They are not case-sensitive. See [Import Tset or Label Statement](#).
For the HSDM digital instruments (HSD1000 and UltraPin800), the maximum number of time sets in a pattern file is as follows:

- | |
|---|
| <ul style="list-style-type: none">Single mode: 64 |
| <ul style="list-style-type: none">Dual mode: 16 |

For the HSDMQ digital instrument (UltraPin1600), the maximum number of time sets in a pattern file is as follows.

- | |
|---|
| <ul style="list-style-type: none">Single mode: 64 |
| <ul style="list-style-type: none">Dual mode: 16 |
| <ul style="list-style-type: none">Quad mode: 8 |

For the HSD4G digital instruments (UltraPin4000 and HPM), the maximum number of time sets in a pattern file is only one in both By1 and By16 mode.

Specifying Tsets

You can specify a tset on each vector in a pattern, including any vector in a vector group. The two ways to specify tsets are:

- Define a \$tset column in the pin list. In this case, there must be a tset name in that column for every vector. See [Pin List](#).
- Specify a tset on selected vectors, to the left of the greater than sign (>), using the keyword \$tset name.

<		>	
	2015 Teradyne, Inc. All rights reserved.		

plvectors.3.09

<	>
---	---

Modules, Vectors, and Vector Data : Symbolic and Numeric Pin Data

Symbolic and Numeric Pin Data

This section covers the following topics:

- Overview of Pin Data
- Symbolic Vector Programming
- Numeric Vector Programming

Note that the `pin_setup` statement determines what pin data states are valid for a pin. See [Pin Setups and Valid Character Sets](#).

<	>	
	2015 Teradyne, Inc. All rights reserved.	

plvectors.3.10

<	>
---	---

Modules, Vectors, and Vector Data : Symbolic and Numeric Pin Data : Overview of Pin Data

Overview of Pin Data

The pin data field defines the vector data to be applied to each column in the pin list. The pin data field is the major component of the vector, which is, in turn, the major part of a pattern file.

Note that the `pin_setup` statement determines what pin data states are valid for a pin. See [Pin Setups and Valid Character Sets](#).

Types of Pin Data

Vector data can be either symbolic or numeric. This section describes both methods:

- Symbolic vector programming uses a set of predefined characters. See [Symbolic Vector Programming](#).
- Numeric vector programming uses binary, octal, hexadecimal, or decimal digits, plus prefixes. See [Numeric Vector Programming](#).

A pin list specifies the radix for each column of data. See [Pin List](#). If no radix is specified in the vector pinlist, the default radix is symbolic.

Overriding the Radix

The radix specified in a pin list defines how data for that pin or group is to be interpreted. A specific data item, however, can override the radix defined in a pin list:

- If a pin list defines a symbolic radix (or uses the default, which is symbolic), a data item in the column prefixed with `.d` (for drive) or `.r` (for receive) is interpreted as a hexadecimal number.
- If a pin list defines a numeric radix, a data item in the column prefixed with `.s` is interpreted as symbolic data. For example, `.sM` and `.sLHHH` indicate symbolic data.

- You can use the notation for filling an entire pin group (a dot followed by a single symbolic character, such as .M or .X) in a column with either a symbolic or numeric radix.

For more on each notation, see [Symbolic Vector Programming](#) and [Numeric Vector Programming](#).

<		>	
		2015 Teradyne, Inc. All rights reserved.	

plvectors.3.11

<

>

Modules, Vectors, and Vector Data : [Symbolic and Numeric Pin Data](#) : Symbolic Vector Programming

Symbolic Vector Programming

This topic includes the following items:

- [Symbolic Character Set](#)
- [Runtime Repeat](#)
- [Symbolic Radix](#)
- [Overriding a Numeric Radix](#)
- [Filling a Pin Group](#)

Note that the `pin_setup` statement determine what pin data states are valid for a pin. See [Pin Setups](#) and [Valid Character Sets](#).

Symbolic Character Set

The pattern compiler accepts these characters for symbolic datacodes:

0	Drive low
1	Drive high
2	Drive very high
L	Expect low
H	Expect high

M	Expect midband
V	Expect valid (high or low)
X	Expect mask
D	Drive ADS (Alternate Data Source)
I	Drive inverse ADS
E	Expect ADS
C	Expect inverse ADS
-	Runtime repeat (hyphen)

Symbolic data characters are not case-sensitive.

Note that some software may refer to datacodes S and R. The S is equivalent to D, and R is equivalent to E.

Runtime Repeat

The runtime repeat character (-), also known as “vector dash,” repeats the pin data from the previously executed vector.

Note that if the first vector of a pattern uses a runtime repeat, then the pins may not repeat the previously executed vector’s data. When starting a new pattern, IG-XL reinitializes the hardware and loses the pin data from the previous pattern burst.

If you need a pattern to retain the previous pattern’s pin data, then you can use the STAY drive format for the specific pin. This format makes the channel retain the drive level from the previous drive cycle. The retained drive level is the voltage level at the end of the previous drive cycle.

Symbolic Radix

An S in a pin list specifies the symbolic radix. If no radix is specified in the pin list, the default radix for the column is symbolic. See [Pin List](#).

Overriding a Numeric Radix

Even if a numeric radix is specified, you can use symbolic for a given vector and pin group by prefixing the symbolic data with .s. For example, .s0 indicates that the 0 is to be interpreted as “Drive low.”

Filling a Pin Group

You can specify a symbolic value for each pin in a pin group. In this case, there must be one symbolic datacode for each pin.

To set an entire pin group to the same symbolic value, use “.” (dot) before a symbolic datacode. This replicates the symbolic data through the current pin group. For example, for a group that contains five pins, these forms are equivalent:

XXXXX .X

You can use this notation for any data item, whether the column’s radix is symbolic or numeric.

<	>	
	2015 Teradyne, Inc. All rights reserved.	

plvectors.3.12

<

>

Modules, Vectors, and Vector Data : Symbolic and Numeric Pin Data : Numeric Vector Programming

Numeric Vector Programming

Numeric Values

In numeric vector programming, values are expressed as either high (1) or low (0) per pin. A prefix identifies whether the value is drive or receive:

.d	Identifies drive data, as in .dF3.
.r	Identifies receive data, as in .rF3.

The .d or .r prefix is required on each numeric data item.

You can use the notation for filling an entire pin group (a dot followed by a single symbolic character, such as .M or .X) in a column with a numeric radix. See [Symbolic Vector Programming](#).

Radix

You must specify a numeric radix for data in the pin list (see [Pin List](#)). If no radix is specified, the default is symbolic. The legal numeric radices are:

X or H	Hexadecimal
Q or O	Octal
D	Decimal
B	Binary

A number of bits equal to the maximum number of channels in the test system can be expressed in all radices except decimal, which is limited to 32 bits.

Overriding a Symbolic Radix

If a pin list defines a column's radix as symbolic (or uses the default, which is symbolic), you can use the .d or .r prefix on an item in the column to override the radix. Such an item is assumed to be in hexadecimal and is interpreted as a hexadecimal number.

LSB and MSB

When using numeric data to program pin groups, the LSB of the data is assigned to the rightmost (last) pin in the pin group. If there are more pins than data bits, low (0) is assumed for the MSBs and programmed into the leftmost (first) pins in the pin group.

For example, suppose pin group ABUS is defined to be pins P0, P1, P2, P3, P4, and P5, in that order, and is programmed with a value of .rA. Pins P0 and P1 get a value of 0 (L), the default for the MSBs when not enough bits are specified in the number. Pins P2 and P4 get a value of 1 (H), and pins P3 and P5 get a value of 0 (L).

.rA = 1010	=	1	0	1	0
		L	L	H	L
		P0	P1	P2	P3
					P4
					P5

Pin Group Example

<	>	
	2015 Teradyne, Inc. All rights reserved.	

plvectors.3.13



Modules, Vectors, and Vector Data : Initialization Pattern

Initialization Pattern

You can specify a pattern that is to be executed before the modules in a pattern file are executed. The initialization pattern can be a pattern module, pattern file, or pattern set. Whenever a pattern that specifies an initialization pattern is executed, the initialization pattern is executed first. This feature lets you create patterns that contain only vectors that actually perform the test, without including initialization vectors in every pattern that requires them.

You specify an initialization pattern using the compiler option `init_pattern`: in the syntax. See [Compiler Control Statements](#). You can also specify an initialization pattern using the Pattern Compiler GUI or command line interface; this lets you specify a single `init_pattern` for many pattern files without having to hardcode the value in the pattern files.

When compiling patterns with an initialization pattern specified, that initialization pattern does not become part of the compiled binary, but is included as part of the pattern set instead. If a pattern is compiled with an initialization pattern specified, then the VMT table contains two entries for that pattern. If a pattern set contains four patterns, and you specify an initialization pattern when compiling each, then the VMT table contains eight entries, and the initialization pattern is burst before each pattern in the set. Since the VMT is used to connect the initialization pattern to the main pattern, you cannot use initialization patterns with the Pattern Sets Sheet burst mode set to `no`, which would stop the PatGen after the initialization pattern and defeat the purpose of the initialization pattern.

The initialization pattern and the pattern to be tested must be able to be executed as a single PatGen burst. Otherwise, an error is returned.

Initialization patterns are valid only with VM modules. SRM modules cannot use initialization patterns because there is no way to guarantee that the initialization pattern and the SRM module can be executed in a single burst.

An initialization pattern cannot be an SRM module. It can call subroutines, but the total amount of SRM required by the initialization pattern and the test pattern must fit in SRM at the same time. An

error is returned if the amount of SRM required is larger than the amount available in hardware. The following table lists possible initialization patterns and comments on which ones are valid.

Initialization Patterns	
Initialization Pattern	Comment
VM module (same as a file containing a single VM module)	No restrictions
File containing multiple VM modules	No restrictions
SRM module (same as a file containing a single SRM module)	Illegal. Cannot guarantee execution of multiple SRM module and test pattern as a single PatGen burst.
File containing multiple SRM modules	Illegal. Cannot guarantee execution of multiple SRM modules as a single PatGen burst.
File containing mix of VM and SRM modules	No restrictions. SRM modules are treated as subroutines.
Pattern set (burst enabled)	No restrictions.
Pattern set (burst disabled)	Illegal. Cannot execute test and init pattern in a single PatGen burst.

You cannot nest initialization patterns. A module that is part of an initialization pattern cannot have its own initialization pattern. This returns an error. In addition, you cannot include an initialization pattern module in the same file as the pattern that uses the initialization pattern (that is, that contains an `init_pattern` compiler option statement). This would result in nesting as well, because the initialization pattern applies to all modules in the pattern file, including the initialization pattern. You can use start and stop labels in patterns that use initialization patterns. The start and stop labels can reside in any module that is part of the initialization or test patterns. Use of start labels can cause the init pattern not to be executed, if the start label is in the test pattern. Similarly, a stop label in the initialization pattern can prevent the test pattern from being executed.

plvectors.3.14

<	>
-----------------	-----------------

Modules, Vectors, and Vector Data : Stop Subroutines

Stop Subroutines

Stop subroutines are vector modules that are executed at the end of a pattern burst. They allow you to set device pins to a known state on completion of the pattern, regardless of how the PatGen stopped. IG-XL calls a stop subroutine when the PatGen encounters a stop condition other than a halt opcode (for example, VMT stop and failing vectors). A stop subroutine must exist for each pattern that is executed. If you do not explicitly create a stop subroutine, IG-XL applies a default stop condition.

UltraFLEX hardware requires two stop subroutines:

- One VM stop subroutine: IG-XL calls the VM stop subroutine when the PatGen encounters a VMT soft stop condition.
- One SRM stop subroutine: IG-XL calls the SRM stop subroutine for all other soft stop conditions.

The following topics are available:

- [Soft Stop Conditions](#)
- [Default Stop Subroutines](#)
- [Creating Stop Subroutines](#)
- [Associating Stop Subroutines with Patterns and Pattern Sets](#)

- Debugging Stop Subroutines
- Stop Subroutines for Pin Setups Lacking Runtime Repeats

Soft Stop Conditions

When a pattern is started, you need not execute all vectors specified in the pattern. You can stop the PatGen on certain conditions that can occur before IG-XL executes all the vectors. A condition other than a halt opcode that causes the PatGen to stop is called a soft stop. Soft stop conditions that result in an SRM stop subroutine call include:

Failing vector	A stop condition that occurs when a vector fails.
History RAM (HRAM) full	A stop condition that occurs when the maximum number of cycles have been stored in HRAM.
Capture Memory (CMEM) full	A stop condition that occurs when the maximum number of cycles have been stored in CMEM.
Stop label	A stop condition that occurs when you start a pattern and stop at a specified stop label. For example, this VBT syntax starts pattern "module1" from vector label "start1" and stops at vector label "stop1": TheHdw.Patterns("module1").Start("start1", "stop1")

If any of these soft stop conditions are met while in either VM or SRM, then the PatGen will call the SRM stop subroutine.

When you burst a VM module, you do not need a halt (and are not allowed to have a return). If a VM module or VM pattern set ends without executing a halt, execution is transferred to the soft stop subroutine. If you have not defined one, the PatGen executes the default stop subroutine.

SRM modules must include a halt or return opcode at the end of the module to ensure that the pattern execution does not run over the end of the module. SRM modules will not automatically call the stop subroutine at the conclusion of the module.

The only soft-stop condition that results in a VM stop subroutine call is a Vector Module Table (VMT) stop, which occurs when the last module in the VMT has been executed.

Default Stop Subroutines

IG-XL provides a default VM and SRM stop subroutine. The default VM stop subroutine consists of a single VM vector which calls the default SRM stop subroutine. The default SRM subroutine consists of a single SRM vector which contains a halt opcode. The pin data that is programmed for both the VM and SRM stop subroutines is runtime repeat (-). The timeset programmed for both of these stop subroutines is timeset address 0.

Note that timeset naming affects timing in a default soft stop subroutine. If a pattern pushes edges into future cycles, then IG-XL cannot guarantee valid timing. Pushing edges into the soft-stop subroutine can return an underflow error because the pushed edges are too close to the soft-stop subroutine's edges.

To prevent these underflow errors in patterns that move edges outside of the cycle in which they are generated, pad these patterns with enough extra cycles to ensure that edges do not move into the soft-stop subroutine. For example, you must pad a cycle that pushes its edges out two cycles with two extra "Don't Care" or "X" cycles so that it does not push edges into the soft-stop subroutine or into any other routine that comes after the module that IG-XL is currently executing. So, if timing matters, write your own soft-stop subroutine rather than using the default.

You need not specify a stop subroutine explicitly. If you do not specify stop subroutines, IG-XL programs default VM and SRM stop subroutines. However, you cannot mix user-defined and default stop subroutines. For instance, you cannot specify a VM stop subroutine without also defining an SRM stop subroutine, or vice versa. Note that this rule does not apply to SRM-only patterns. If the pattern does not contain any VM modules, then you do not need to include a VM stop subroutine.

Creating Stop Subroutines

When executing patterns that do not contain a halt opcode, IG-XL creates a stop subroutine by default. If you do not want to use this default stop subroutine, you can create your own stop subroutine.

To create a stop subroutine, include a stop subr label in a pattern module. In an SRM module, this label can appear on any vector. In a VM module, this label must appear on the first vector. Only one stop subroutine label is allowed per module. Stop subroutine labels are global which means that a given named label can only be defined in one module. Multiple VM and SRM stop subroutines can be used with each test program.

Pins, pin setups, and PatGen opcode mode for the stop subroutines and the pattern must be the same because your test program must be able to execute stop subroutines in a single PatGen burst with their associated patterns. IG-XL returns a validation error if the pattern and stop subroutines are inconsistent.

Stop subroutine modules must be loaded by the test program. To do this, list the names of the stop subroutines on the Pattern Subroutine sheet.

Note that if you need to load a pattern file that contains only subroutines that are not referenced as arguments in a test instance, then you should use the Pattern Subroutines sheet to load those subroutines. When another pattern that is included in the job uses the pattern files listed on this sheet, IG-XL loads the subroutine pattern module into VM at validation time, moves it to SRM at start time, and unloads it from SRM after the pattern has run. IG-XL does not load any subroutine pattern files that are not used by another pattern.

IG-XL does not enforce the VM minimum size restriction for VM stop subroutines.

The following examples show a stop subroutine in SRM and VM modules.


```
import tset ts0;
// Example SRM stop subroutine
srm_vector srm_stop_subroutine ( $tset, pin0 )
{
stop subr srmStopLabel:
halt      > ts0  0 ;
}
```

```
// Example VM stop subroutine
vm_vector vm_stop_subroutine ( $tset, pin0 )
{
stop subr vmStopLabel:
call srmStopLabel    > ts0  0 ;
}
```

Associating Stop Subroutines with Patterns and Pattern Sets

Each pattern file or module that is executed in the test program must have a stop subroutine associated with it. You can explicitly specify the stop subroutines or use default subroutines. You cannot simply associate a stop subroutine with a pattern set, however, because you can execute the individual patterns of the set using VBT or PatternTool.

To associate a stop subroutine with a pattern, import the VM and SRM stop subroutine label names in the pattern file. If more than one module is contained in the pattern file, IG-XL associates the stop subroutines with each module in the file. You can either use one VM and one SRM stop subroutine for all patterns, or create several VM and SRM stop subroutines for different patterns or groups of patterns in the test program.

When you execute a pattern set in burst mode, you must specify the same VM and SRM stop subroutines for all of the set's patterns. Alternatively, you can use the default stop subroutines by not defining any VM or SRM stop subroutines. IG-XL returns an error if you mix patterns that have explicitly specified stop subroutines with those that do not.

The following example shows how to use import statements in a pattern to include stop subroutines.

```
import tset ts0;
import stop subr vmStopLabel;
import stop subr srmStopLabel;

vm_vector module0 ($tset, pin0)
{
    > ts0  0 ;
    > ts0  0 ;
}
```

```
> ts0 0 ;
```

```
...  
}
```

Debugging Stop Subroutines

You can display SRM stop subroutines in HRAM PatternTool for debugging purposes. The Vector/Cycle column shows the module name containing the stop subroutine. PatternTool also displays all other data captured in the HRAM.

TSet	Command	Label	Vector/Cycle	SyncCyc	TimeCnt	ModCnt
ts0	set		P.0/0(mod0)	0	0	0
ts1	push_loop		P.1/1(mod0)	1	2000	1
ts2	reload_ctrs		P.2/2(mod0)	2	4000	2
ts2	nop		P.3/3(mod0)	3	6000	3
ts1	nop	subr1	P.0/4(srm_stop_subroutine)	4	8000	4
ts0	nop		P.1/5(srm_stop_subroutine)	5	10000	5
ts2	halt		P.2/6(srm_stop_subroutine)	6	12000	6

SRM Stop Subroutines in the HRAM PatternTool

HRAM PatternTool displays the default VM stop subroutine with the same module name as the last VM module that was executed with a vector offset of 0. Note that the VM stop subroutine module name does not appear in the Vector/Cycle column. Note also that HRAM PatternTool shows the cycles created by the VM stop subroutine as part of the last module. This is shown in the following figure.

HSDM HRAM (Wiggle_module,FailWiggle)

TSet	Label	Vector/Cycle	SyncCyc	TimeCnt	ModCnt	VecRep
FastTS		P.1/1(Wiggle_module)	1	40	1	0
FastTS		P.2/2(Wiggle_module)	2	80	2	0
FastTS		P.3/3(Wiggle_module)	3	120	3	0
FastTS		P.4/4(Wiggle_module)	4	160	4	0
FastTS		P.5/5(Wiggle_module)	5	200	5	0
FastTS	start	P.0/6(FailWiggle)	6	240	0	0
FastTS		P.1/7(FailWiggle)	7	280	1	0
FastTS		P.2/8(FailWiggle)	8	320	2	0
FastTS		P.3/9(FailWiggle)	9	360	3	0
FastTS		P.4/10(FailWiggle)	10	400	4	0
FastTS		P.5/11(FailWiggle)	11	440	5	0
FastTS	start	P.0/12(FailWiggle)	12	480	0	0
FastTS		P.0/13(tl_srm_stop_subr)	13	520	1	0

Vector Offset at 0

VM Stop Subroutine

SRM Stop Subroutine

VM Stop Subroutines in HRAM PatternTool

Stop Subroutines for Pin Setups Lacking Runtime Repeats

The SRM and VM default stop subroutines use a vector dash (-) to program a runtime repeat for all pins so that they hold the state they had at the last vector of the pattern. Pins that use certain pin setups or instruments do not support runtime repeats. The drivers for all such pins go to either the HiZ state or drive VT, depending on the value of the DriverMode parameter for the current level context. The following pin setups and instruments do not support runtime repeat:

- Pin setups 2x_input, 2x_output, 2x_agp, 2x, and window

- DSSC instruments
- MTO instrument (when `allow_mto_dash = no`, which does not support a runtime repeat)

The default stop subroutine programs an XX pin state for 2x, window, DSSC, and MTO pins. If you want the pin state in the stop subroutine to be the same as it was in the last vector of the pattern, you have two options:

- Insert a halt opcode at the end of the pattern.

This causes the PatGen to stop on the vector with the halt opcode, and IG-XL does not call the default stop subroutine. This option is not recommended if you are using the patterns in pattern sets, because the halt opcode prevents the pattern set from executing correctly.

- Define a custom VM and SRM stop subroutine that preserves the driver state.

For more information, see [Creating Stop Subroutines](#). To preserve the driver state from the last vector in the pattern, program a Stay format for the 2x, window, DSSC, and MTO pins for the time set used in the stop subroutines.

The following shows the implementation of the second option, using a stop subroutine and 2x pin setup:

```
import tset ts_stay;
pin_setup = {
    pin0 2x;
}
// Example SRM stop subroutine
srm_vector srm_stop_subroutine ( $tset, pin0 )
{
    stop subr srmStopLabel:
    halt                > ts_stay 0
                        0;
}
// Example VM stop subroutine
vm_vector vm_stop_subroutine ( $tset, pin0 )
{
    stop subr vmStopLabel:
    halt                > ts_stay 0
```

```
0;  
}
```

The following pattern refers to the stop subroutine:

```
import tset ts0;  
import stop subr vmStopLabel;  
import stop subr srmStopLabel;  
pin_setup = {  
    pin0 2x;  
}  
vm_vector mod1 ( $tset, pin0 )  
{  
    > ts0 0  
    0;  
    > ts0 0  
    0;  
// Remainder of the pattern file  
...  
}
```

The stop subroutine must be included on the Pattern Subroutine sheet.
The stop subroutine uses the time set ts_stay. On a Time Sets (Basic) sheet, the time set should use a Stay format:

Time Sets (Basic)									
Timing Mode: Single		Master Timeset Name:							
Time Domain:									
Time Set	Cycle		Pin/Group		Data		Drive		
	Period	Name	Clock Period	Setup	Src	Fmt	First	Return First	Second
ts0	10.E-09	pin0		2X	PAT	NR-2X	2.E-09		8.E-09
ts_stay	10.E-09	pin0		2X	PAT	STAY-2X			

Time Sets (Basic) Sheet with Stay Format

<

>

2015 Teradyne, Inc. All rights reserved.

plvectors.3.15

<	>
-----------------	-----------------

Modules, Vectors, and Vector Data : Keepalive Subroutines

Keepalive Subroutines

Keepalive subroutines allow test system hardware to be modified while the PatGen is running. The additional attribute of a keepalive subroutine is that IG-XL knows what resources the subroutine uses and will not overwrite those resources during the modification of test system memories. The resources that are protected are the locations in the SRM itself which store the keepalive subroutine and the timing memory locations (timesets) used during execution of the keepalive subroutine. For more information on the keepalive pattern file syntax and programming, see [Keepalive](#) in IG-XL Programming Concepts.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--